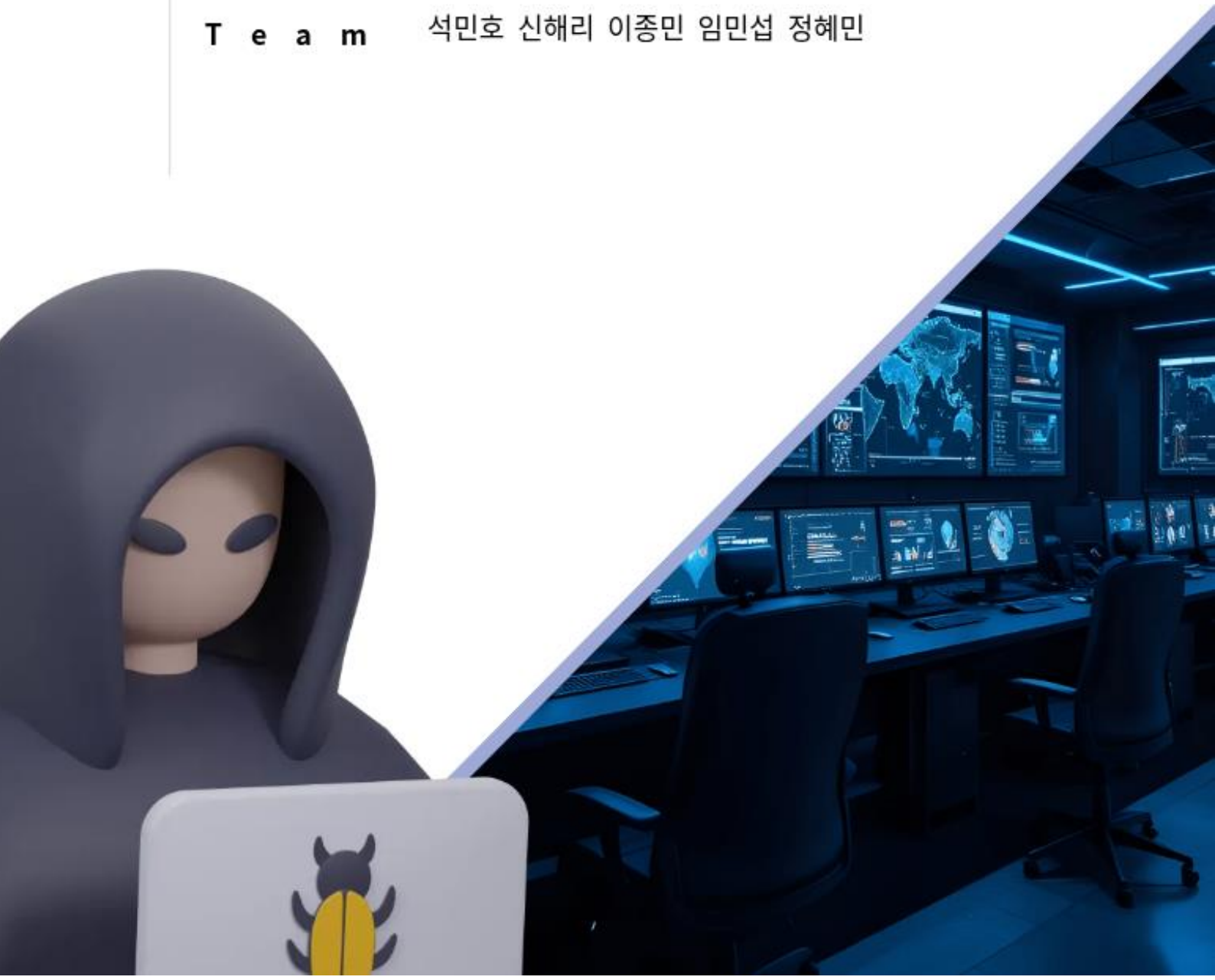
TERRAFORM을 활용한 AWS 기반 WORDPRESS 구축
-고가용성 AUTOSCALING과 WAF 보안 강화-

보고서

Period 2025.12.17 ~ 2026.01.02

Team 석민호 신해리 이종민 임민섭 정혜민

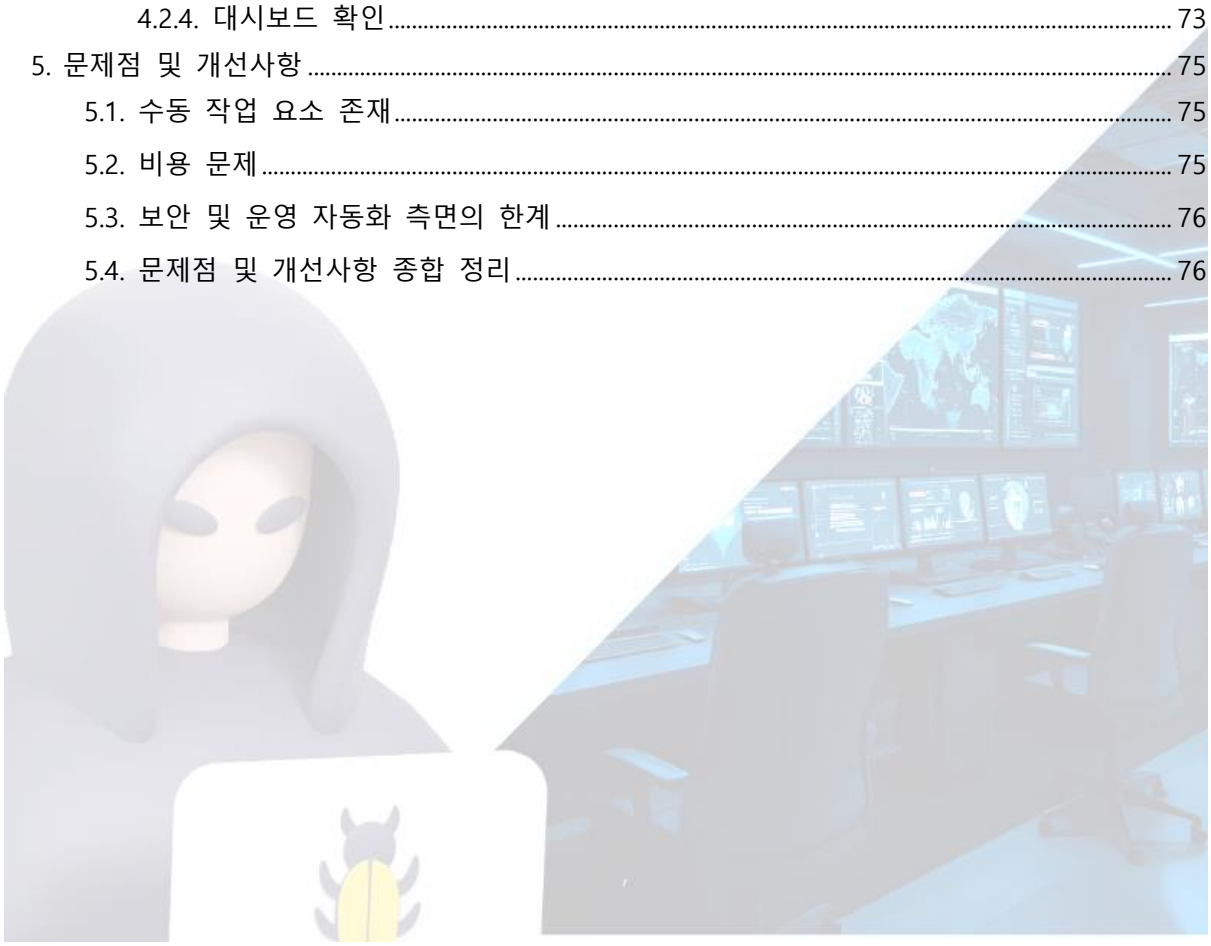


목차

1. 서론 7
1.1. 프로젝트 배경 7
1.2. 프로젝트 목적 8
1.2.1. IaC 기반 인프라 자동화 8
1.2.2. 고가용성 및 트래픽 분산 구성 8
1.2.3. 보안 통제 적용 8
1.3. 기대효과 9
2. 환경 구성 10
2.1. AWS 인프라 아키텍처 10
2.2. AWS 인프라 구축 절차 11
2.3. 개발 및 운영 도구 구성 11
3. 구축 과정 12
3.1. Terraform 모듈 구조 12
3.1.1. Terraform 기본 설정 13
3.1.2. YAML 기반 선언적 변수 관리 14
3.1.3. S3 기반 Terraform Remote Backend 구성 15
3.2. 네트워크 인프라 구성 18
3.2.1. Region 생성 18
3.2.2. VPC 생성 19
3.2.3. Subnet 생성 20
3.3. 인터넷 및 라우팅 구성 23
3.3.1. 라우팅 테이블 23
3.3.2. Internet Gateway 25
3.3.3. Route53 26
3.4. EC2 및 웹 서비스 구성 28
3.4.1. EC2 28
3.5. AMI 기반 표준 이미지 생성 31
3.5.1. 웹서비스 생성 및 AMI 이미지 생성 및 연결 31
3.5.2. Launch Template 구성 33
3.6. 데이터베이스 구성 36
3.6.1. RDS 생성 36
3.6.2. 데이터베이스 생성 후 RDS 스냅샷 생성 및 연결 39
3.7. 고가용성 구성 41

Copyright ⓒ 2025 Team_JS. All rights reserved.

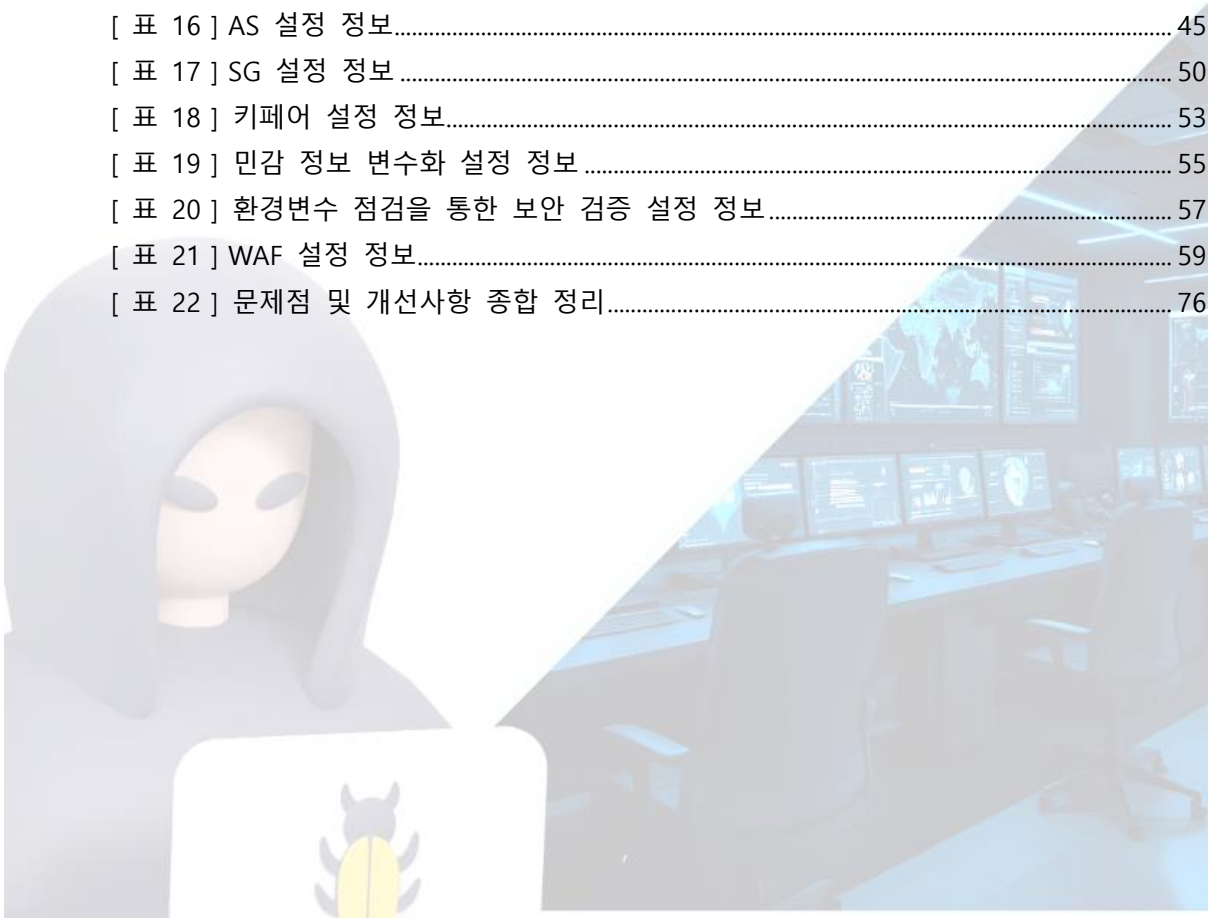
3.7.1. Target Group 41
3.7.2. Load Balancer 43
3.7.3. Auto Scalling 45
3.8. 보안 기본 구성 50
3.8.1. Security Group 생성 50
3.8.2. 키페어 생성 53
3.9. 보안 및 트래픽 제어 55
3.9.1. 민감 정보 변수화 및 보안 관리 55
3.9.2. 환경변수 점검을 통한 보안 검증 57
3.9.3. WAF 정책 및 규칙 설정 59
4. 성능 테스트 65
4.1. 인프라 구성 결과 검증 65
4.2. WAF로 공격 차단 및 로그 확인 67
4.2.1. BruteForce 차단 67
4.2.2. SQLiRule 차단 69
4.2.3. CommonRule 차단 71
4.2.4. 대시보드 확인 73
5. 문제점 및 개선사항 75
5.1. 수동 작업 요소 존재 75
5.2. 비용 문제 75
5.3. 보안 및 운영 자동화 측면의 한계 76
5.4. 문제점 및 개선사항 종합 정리 76



Copyright ⓒ 2025 Team_JS. All rights reserved.

표

[표 1] 개발 및 운영 도구 구성 11
[표 2] Terraform Provider 설정 정보 13
[표 3] YAML 기반 선언적 변수 관리 설정 정보 14
[표 4] S3 활용 Remote Backend 설정 정보 15
[표 5] Region 설정 정보 18
[표 6] VPC 네트워크 설정 정보 19
[표 7] subnet 생성 정보 20
[표 8] 라우팅 테이블 설정 정보 23
[표 9] Internet Gateway 설정 정보 25
[표 10] Route53 설정 정보 26
[표 11] ec2 설정 정보 28
[표 12] Template 설정 정보 33
[표 13] RDS 설정 정보 36
[표 14] TG 설정 정보 41
[표 15] LB 설정 정보 43
[표 16] AS 설정 정보 45
[표 17] SG 설정 정보 50
[표 18] 키페어 설정 정보 53
[표 19] 민감 정보 변수화 설정 정보 55
[표 20] 환경변수 점검을 통한 보안 검증 설정 정보 57
[표 21] WAF 설정 정보 59
[표 22] 문제점 및 개선사항 종합 정리 76



Copyright ⓒ 2025 Team_JS. All rights reserved.

그림

[그림 1] Terraform 기반 AWS 인프라 구성도 10
[그림 2] Terraform 기반 AWS 인프라 구축 절차 11
[그림 3] 테라폼 모듈 구조도 12
[그림 4] 버킷 구축 결과 17
[그림 5] region 설정 결과 18
[그림 6] vpc 생성 결과 20
[그림 7] subnet 생성 결과 22
[그림 8] 라우팅 테이블 생성 결과 24
[그림 9] Internet Gateway 생성 결과 25
[그림 10] Route53 생성 결과 27
[그림 11] EC2 생성 결과 30
[그림 12] http 설치 31
[그림 13] 패키지 설치 31
[그림 14] tar 파일 해제 32
[그림 15] 소유자 apache 변경 및 httpd start 32
[그림 16] wordpress 하위 파일 html로 이동 32
[그림 17] 이미지 생성 과정 32
[그림 18] 이미지 생성 확인 33
[그림 19] Template 생성 결과 35
[그림 20] RDS 생성 결과 38
[그림 21] Mariadb 설치 39
[그림 22] 데이터베이스 초기 생성 과정 39
[그림 23] 스냅샷 생성 40
[그림 24] 스냅샷 생성 완료 40
[그림 25] TG 생성 결과 43
[그림 26] LB 생성 결과 44
[그림 27] AS 생성 결과 49
[그림 28] AS 생성 결과 인스턴스 관리 창 49
[그림 29] SG 생성 결과 52
[그림 30] 키페어 생성 결과 54
[그림 31] Waf 구축 결과 64
[그림 33] top 명령어 실행 결과 65
[그림 33] CloudWatch > Metrics 의 CPU 사용률 그래프 66
[그림 34] CPU 부하에 따른 자동 확장(Scale-out) 수행 결과 66
[그림 35] BruteForece 공격 코드 67

Copyright ⓒ 2025 Team_JS. All rights reserved.

[그림 36] BruteForece 공격결과 68
[그림 37] BrutoForce 로그 확인 68
[그림 38] SQL 공격 코드 69
[그림 39] SQL 공격 결과 70
[그림 40] SQL 로그 확인 70
[그림 41] CommonRule 공격 코드 71
[그림 42] CommonRule 공격 결과 72
[그림 43] CommonRule 로그 확인 72
[그림 44] WAF 및 쉴드 > 보호 팩(웹 ACL) > seoul-waf-js > 대시보드의 국가별 공격 73
[그림 45] WAF 및 쉴드 > 보호 팩(웹 ACL) > seoul-waf-js > 대시보드에서의 공격유형 74

Copyright ⓒ 2025 Team_JS. All rights reserved.

코드

[코드 1] Terraform Provider 기본 설정 코드_ 🗀 **seoul/main.tf** 13
[코드 2] YAML 기반 선언적 변수 관리 구성 코드_ 🗀 **seoul/main.tf** 14
[코드 3] S3 기반 협업 구성 코드_ 🗀 **modules/terraform-backend/main.tf** 16
[코드 4] region 설정 코드_ 🗀 **seoul/main.tf** 18
[코드 5] region 변수 관리 코드_ 🗀 **seoul/variables.yml** 18
[코드 6] vpc 생성 코드_ 🗀 **modules/vpc/main.tf** 19
[코드 7] vpc 변수 관리 코드_ 🗀 **seoul/variables.yml** 19
[코드 8] subnet 설정 코드_ 🗀 **modules/vpc/main.tf** 21
[코드 9] subnet 변수 관리 코드_🗀 **seoul/variables.yml** 22
[코드 10] 라우팅 테이블 설정 코드_🗀 **modules/vpc/main.tf** 24
[코드 11] Internet Gateway 설정 코드_🗀 **modules/vpc/main.tf** 25
[코드 12] Route53 설정 코드_ 🗀 **modules/route53/main.tf** 26
[코드 13] ec2 생성 코드_ 🗀 **modules/ec2/main.tf** 29
[코드 14] ec2 변수 관리 코드_ 🗀 **seoul/variables.yml** 30
[코드 15] Template 생성 코드_ 🗀 **modules/launch_template/main.tf** 34
[코드 16] Template 변수 관리 코드_ 🗀 **seoul/variables.yml** 35
[코드 17] RDS 생성 코드_ 🗀 **modules/rds/main.tf** 37
[코드 18] RDS 변수 관리 코드_🗀 **seoul/variables.yml** 38
[코드 19] TG 설정 코드_ 🗀 **modules/target-group/main.tf** 42
[코드 20] TG 변수 관리 코드_ 🗀 **seoul/variables.yml** 42
[코드 21] LB 설정 코드_🗀 **modules/load-balancer/main.tf** 44
[코드 22] LB 변수 관리 코드_ 🗀 **seoul/variables.yml** 44
[코드 23] AS 설정 코드_ 🗀 **modules/auto-scaling/main.tf** 47
[코드 24] AS 변수 관리 코드_ 🗀 **seoul/variables.yml** 48
[코드 25] SG 설정 코드_ 🗀 **modules/security-group/main.tf** 51
[코드 26] SG 변수 관리 코드_🗀 **seoul/variables.yml** 52
[코드 27] 키페어 설정 코드_ 🗀 **modules/security-group/main.tf** 54
[코드 28] 키페어 변수 관리 코드_🗀 **seoul/variables.yml** 54
[코드 29] 민감 정보 변수화 설정 코드_🗀 **seoul/main.tf** 56
[코드 30] 환경변수 점검 스크립트_🗀 **seoul/check-env.ps1** 58
[코드 31] LB에 WAF 연결 코드_ 🗀 **modules/load-balancer/main.tf** 59
[코드 32] 단일 Rule Group_🗀 **modules/waf/main.tf** 60
[코드 33] 다중 Rule Group_🗀 **modules/waf/main.tf** 61
[코드 34] Rate-Based Rule _🗀 **modules/waf/main.tf** 62
[코드 35] WAF 변수 관리 코드_ 🗀 **seoul/variables.yml** 63

Copyright ⓒ 2025 Team_JS. All rights reserved.

1. 서론

1.1. 프로젝트 배경

최근 웹 서비스 환경은 보안 위협의 급증과 인프라 운영 복잡성 확대로 인해 기존 수동 중심 운영 방식의 한계를 직면함.

- **수동 설정에 따른 운영 리스크:** Gartner 조사에 따르면 인프라 장애의 약 80%가 운영자의 수동 설정 오류에서 기인함. 이는 환경 불일치 및 서비스 장애로 직결되어 시스템 신뢰도를 저하시키는 주요 원인이 됨.
- **유연한 대응 체계의 부재:** 트래픽 변동이 잦은 환경에서 수동 기반의 자원 증설 및 장애 대응은 신속성이 결여되며, 결과적으로 성능 저하, 복구 지연 등의 리스크가 지속적으로 발생함.
- **보안 위협의 급증:** 전 세계 웹 공격이 매년 60% 이상 증가하고 있으며, SQL Injection 및 XSS 등 애플리케이션 계층을 겨냥한 공격이 지능화됨. 이에 따라 단순 서버 구축을 넘어선 통합적인 보안 체계 마련이 필수적임.

결론적으로, 인프라 운영의 일관성을 확보하고 장애 발생을 최소화하며, 보안 위협에 선제적으로 대응할 수 있는 자동화 기반의 지능형 인프라 운영 체계 도입이 요구됨.

Copyright ⓒ 2025 Team_JS. All rights reserved.

1.2. 프로젝트 목적

본 프로젝트는 Terraform 기반의 자동화된 AWS 인프라 운영 환경을 적용함으로써, 다음과 같은 목적을 가진다.

1.2.1. IaC 기반 인프라 자동화

- **수동 설정 제거를 통한 운영 리스크 감소:** 인프라 자원을 코드로 관리함으로써 수동 설정 과정에서 발생할 수 있는 설정 누락, 오입력, 환경 간 설정 불일치 문제를 사전에 방지하는 것을 목표로 함.
- **환경 구성 변경 및 재배포 작업의 단순:** Terraform IaC 와 환경별 변수 파일(YAML)을 통해 변수 값 변경만으로 환경 구성을 관리하고, 동일 인프라를 반복적으로 생성·수정할 수 있는 운영 체계를 목표로 함.
- **모듈화 기반 인프라 관리 효율성 확보:** 인프라를 모듈 단위로 관리하여 확장 및 변경 시 코드 수정 범위를 최소화하는 것을 목표로 함.

1.2.2. 고가용성 및 트래픽 분산 구성

- **트래픽 분산을 통한 단일 장애 지점 제거:** 외부 요청을 단일 엔드포인트에서 수신하여 다수의 백엔드 인스턴스로 분산함으로써 특정 자원 장애로 인한 서비스 중단을 방지하는 것을 목표로 함.
- **정상 자원 대상 트래픽 전달:** 백엔드 인스턴스의 상태를 지속적으로 확인하여 정상 상태의 자원에만 트래픽을 전달함으로써 서비스 안정성을 유지하는 것을 목표로 함.
- **트래픽 변화 대응을 위한 자동 확장**: 트래픽 변화에 따라 자원을 자동으로 확장·축소하여 수동 대응 부담을 줄이는 운영 체계를 목표로 함.

1.2.3. 보안 통제 적용

- **애플리케이션 계층 보안 강화:** 웹 요청 단계에서 발생하는 SQL Injection, XSS 등 애플리케이션 계층 공격을 사전에 탐지·차단하여 위협 확산을 방지하는 것을 목표로 함.
- **네트워크 접근 제어 강화:** Security Group 을 활용하여 웹, 애플리케이션, 데이터베이스 계층 간 접근 권한을 분리하고, 최소 권한 원칙에 기반한 자원 접근 통제를 적용하는 것을 목표로 함.

Copyright ⓒ 2025 Team_JS. All rights reserved.

1.3.기대효과

본 프로젝트를 통해 Terraform 기반의 자동화된 AWS 인프라 운영 환경을 적용함으로써, 다음과 같은 효과를 기대할 수 있음.

1. **고가용성 및 트래픽 분산 구성:** 인프라를 코드로 관리함으로써 수동 설정에 따른 인적 오류를 줄이고, 환경 간 구성 일관성과 재현성을 확보하여 안정적인 인프라 운영이 가능함.
2. **고가용성 및 트래픽 분산 구성:** 부하 분산 및 자동 확장 구조를 통해 트래픽 변동이나 개별 자원 장애 상황에서도 서비스 중단 없이 안정적인 운영 환경을 유지할 수 있음.
3. **보안 통제 적용:** 애플리케이션 및 네트워크 계층에 대한 보안 통제를 적용하여 웹 공격 및 비인가 접근을 효과적으로 차단하고 전반적인 보안 수준을 향상시킬 수 있음.

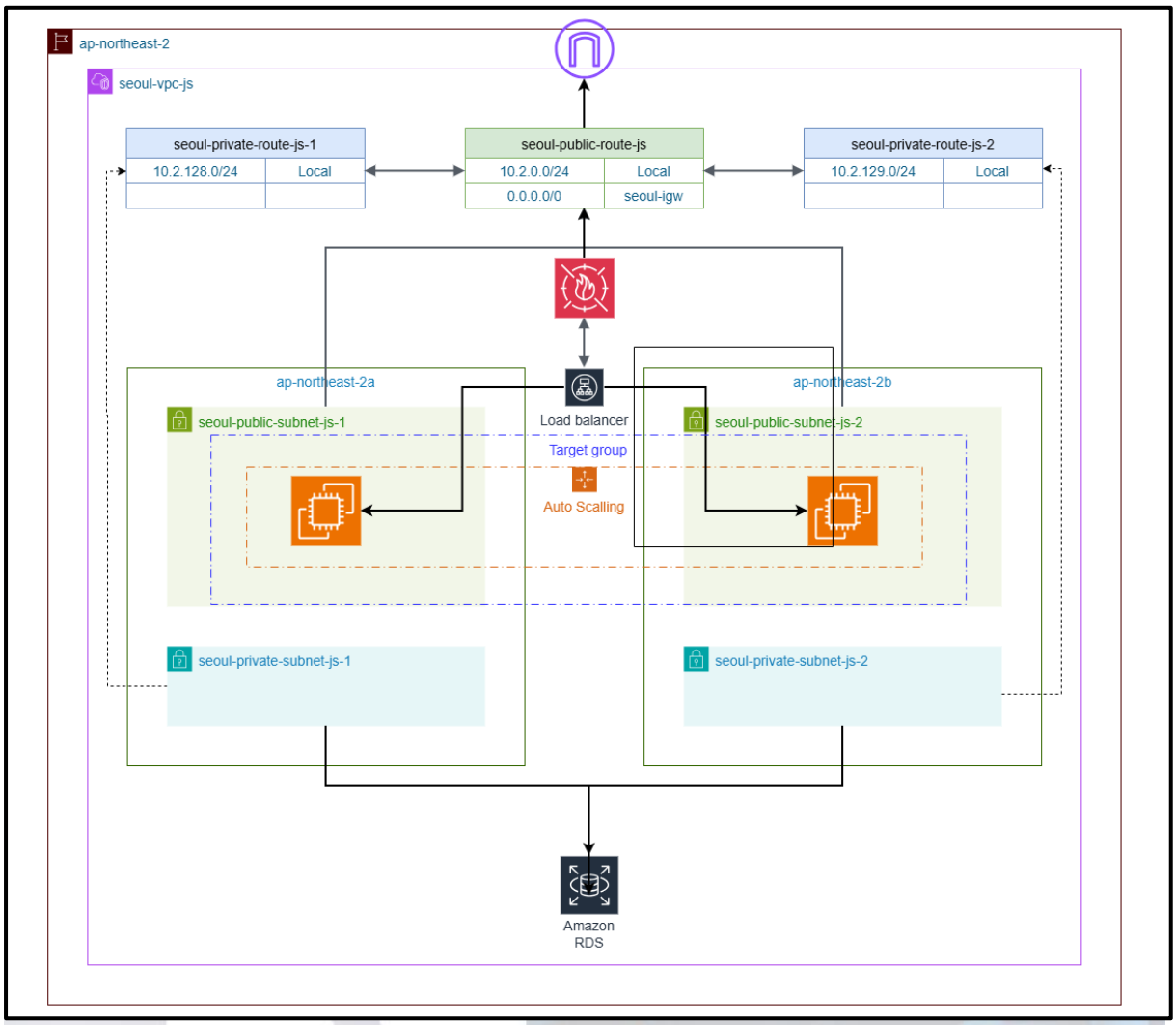
Copyright ⓒ 2025 Team_JS. All rights reserved.

2. 환경 구성

2.1. AWS 인프라 아키텍처

본 구성도는 Terraform 기반으로 구축한 AWS 인프라의 전체 구조를 나타냄.

1. VPC 내에 퍼블릭·프라이빗 서브넷을 가용 영역별로 분리하여 구성함.
2. 외부 요청은 Application Load Balancer를 통해 수신한 후 Target Group과 Auto Scaling Group으로 분산 처리되도록 설계함.
3. 애플리케이션 서버는 다중 가용 영역에 배치되어 서비스 연속성을 유지할 수 있음.
4. 데이터베이스는 프라이빗 서브넷에 위치한 RDS를 통해 보안성과 안정성을 확보함.

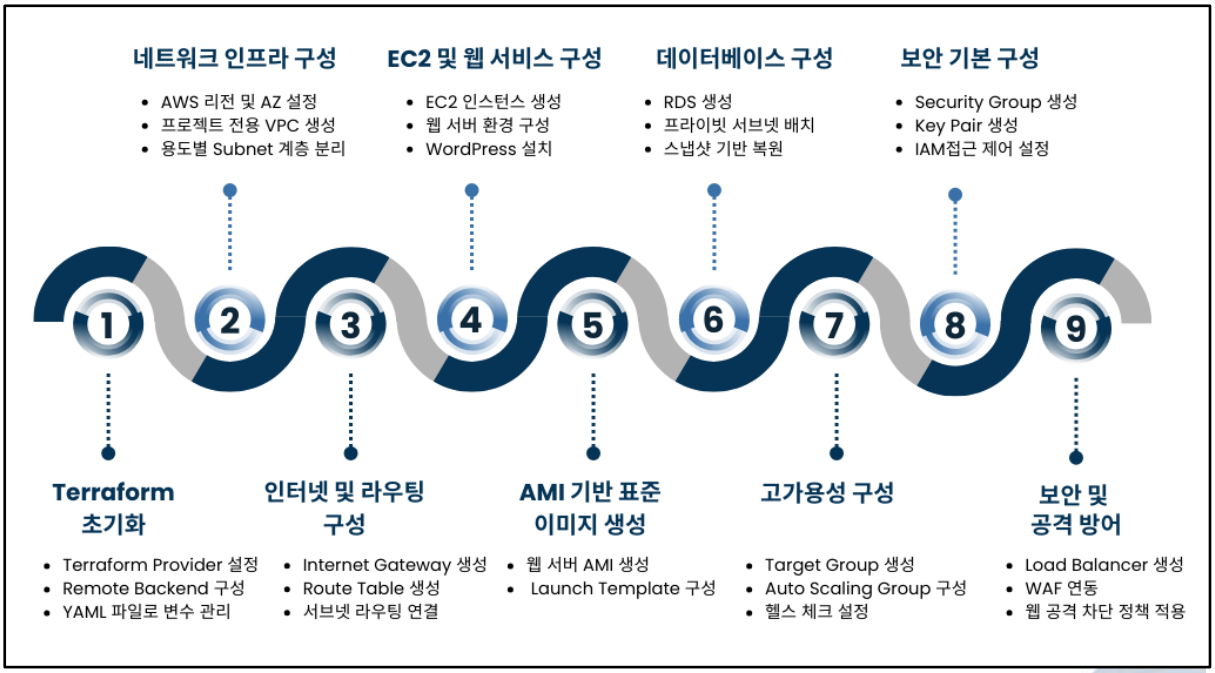


[그림 1] Terraform 기반 AWS 인프라 구성도

Copyright ⓒ 2025 Team_JS. All rights reserved.

2.2. AWS 인프라 구축 절차

본 구성도는 Terraform을 활용한 실제 AWS 인프라 구축 절차를 기반으로 작성되었으며, 이후의 구축 과정 설명은 본 구성도의 순서를 기준으로 단계별로 기술함.



[그림 2] Terraform 기반 AWS 인프라 구축 절차

2.3. 개발 및 운영 도구 구성

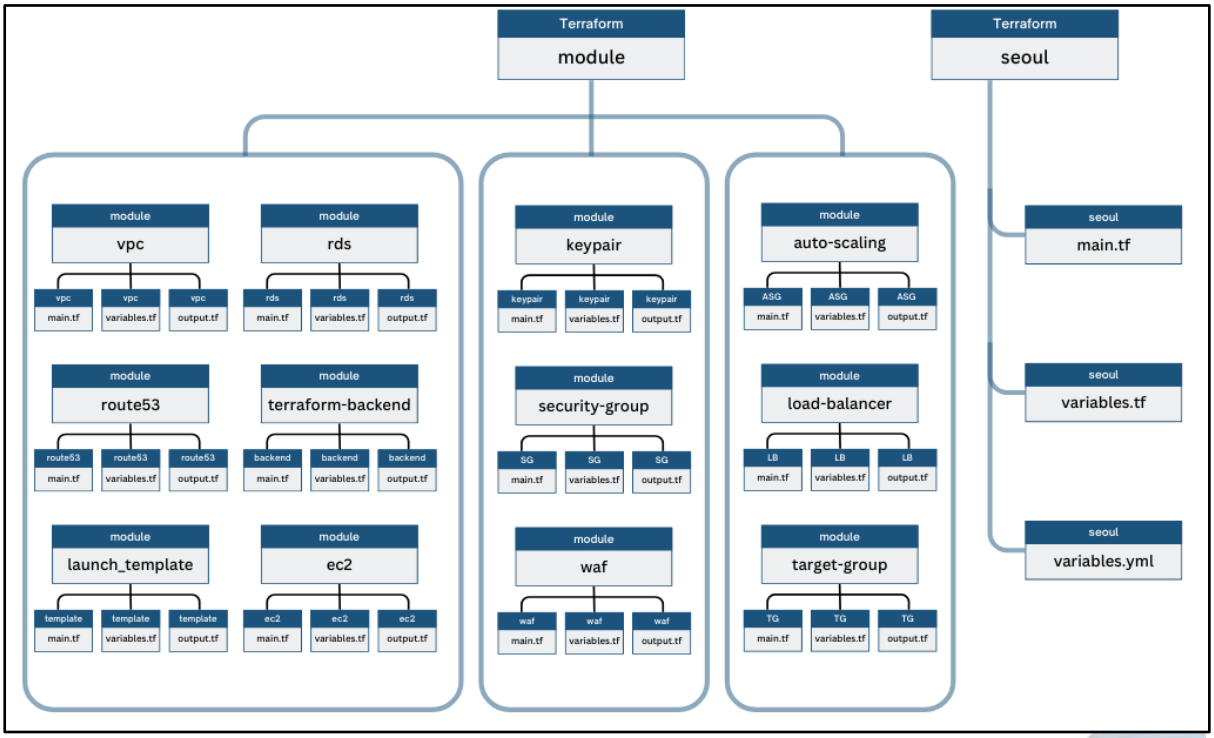
구분	도구명 (Tool)	주요 용도 및 역할
로컬 작업 환경	Windows PowerShell /CMD	Terraform 및 AWS CLI 명령어 실행
코드 편집기	Visual Studio Code	Terraform 코드 및 YAML 작성 및 편집
원격 접속 도구	Xshell	AWS EC2(Linux) 서버 원격 접속 및 관리
원격 접속	Terraform	AWS 인프라를 자동 생성 및 관리
클라우드 플랫폼	AWS	인프라 리소스를 제공하는 클라우드 환경
인프라 자동화 도구	Linux (Amazon Linux 2023)	웹 서비스(WordPress) 구동 환경

[표 1] 개발 및 운영 도구 구성

Copyright ⓒ 2025 Team_JS. All rights reserved.

3. 구축 과정

3.1. Terraform 모듈 구조



[그림 3] 테라폼 모듈 구조도

Copyright ⓒ 2025 Team_JS. All rights reserved.

3.1.1. Terraform 기본 설정

[설계 의도 요약]

Terraform 실행 환경 차이로 인한 배포 오류를 방지하고자 함.

이를 위해 Terraform 버전과 AWS Provider 버전을 명시적으로 고정하여 모든 환경에서 동일한 조건으로 코드가 실행되도록 구성함.

그 결과 환경별 실행 결과 편차를 줄이고, 예측 가능한 인프라 배포가 가능함.

[관련 요청사항 번호: S-04]

항목	내용
리소스 유형	Terraform (IaC)
Terraform 버전	1.0.X 계열
Provider 정보	hashicorp/aws (~> 6.0)

[표 2] Terraform Provider 설정 정보

[상세 구현 코드]

```
# ============================================================
# Terraform 및 Provider 기본 설정
# - 버전 고정으로 실행 환경 일관성 확보
# ============================================================
terraform {
  required_version = "~> 1.0"
  required_providers {
      aws = {
              source  = "hashicorp/aws"
              version = "~> 6.0
      }
    }
}
```

[코드 1] Terraform Provider 기본 설정 코드_ 📂 **seoul/main.tf**

Copyright ⓒ 2025 Team_JS. All rights reserved.

3.1.2. YAML 기반 선언적 변수 관리

[설계 의도 요약]

인프라 설정 값 변경 시 코드 수정으로 발생할 수 있는 오류를 줄이고자 함.

이를 위해 인프라 로직과 설정 데이터를 분리하고, YAML 파일에 변수 값을 정의하여 Terraform에서 이를 로드하도록 구성함.

그 결과 변수 값 변경만으로 환경 구성이 가능하며, 설정 관리의 가독성과 재사용성이 향상됨.

[관련 요청사항 번호: S-04]

항목	내용
변수 관리 방식	YAML 파일 기반 변수 관리
변수 로딩 방식	yamldecode(), file()
적용 범위	VPC, Subnet, 보안 설정 등
기대 효과	환경 분리, 재사용성, 가독성 향상

[표 3] YAML 기반 선언적 변수 관리 설정 정보

[상세 구현 코드]

```
# ============================================================
# YAML 기반 변수 로드 및 병합
# - 인프라 설정 값을 코드와 분리하여 YAML 파일로 관리
# - 환경별 변수 변경 시 Terraform 코드 수정 최소화
# ============================================================
locals {
  yaml_vars = yamldecode(file("./variables.yml"))
}
```

[코드 2] YAML 기반 선언적 변수 관리 구성 코드_ 📂 **seoul/main.tf**

Copyright ⓒ 2025 Team_JS. All rights reserved.

3.1.3. S3 기반 Terraform Remote Backend 구성

[설계 의도 요약]

Terraform 상태 파일을 로컬에서 관리하면서 발생할 수 있는 유실 및 충돌 위험을 제거하고자 함.

이를 위해 상태 파일을 S3 에 원격 저장하고, 버전 관리와 암호화를 적용하며 DynamoDB 를 이용한 잠금 구조를 구성함.

그 결과 다중 사용자 환경에서도 상태 충돌 없이 안정적인 IaC 운영이 가능함.

[관련 요청사항 번호: S-04]

항목	내용
리소스 유형	S3 Bucket & DynamoDB
버킷 명	seoul-s3-js-tfstate
키 경로	seoul/terraform.tfstate
Region	ap-northeast-2
버전 관리	활성화 (Enabled)
암호화	SSE-S3 방식 적용 (AES256)
상태 잠금	DynamoDB 테이블 구성(backend 연동 시 locking 적용)

[표 4] S3 활용 Remote Backend 설정 정보

Copyright ⓒ 2025 Team_JS. All rights reserved.

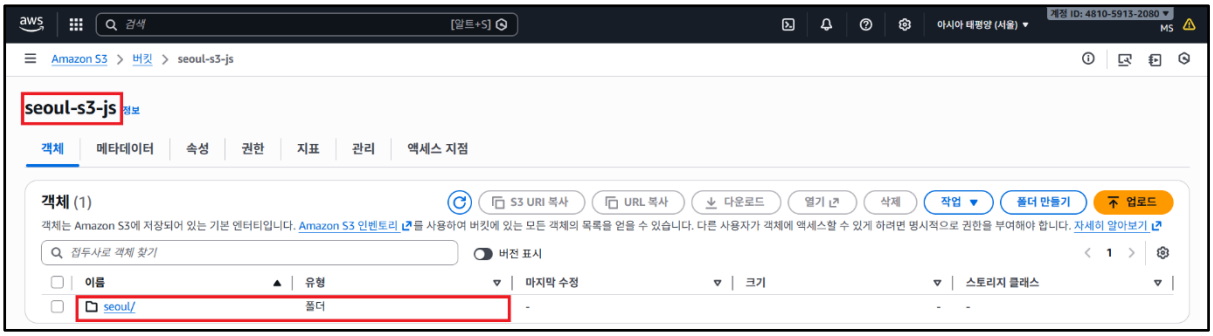
[상세 구현 코드]

```
# ------------------------------------------------------------
# S3 Bucket Encryption
# state 파일 암호화를 통한 보안 강화
# ------------------------------------------------------------
resource "aws_s3_bucket" "terraform_state" {
  bucket = var.bucket_name
  tags = {
    Name        = var.bucket_name
    Purpose     = "Terraform State Storage"
    Environment = "shared"
  }
}
# S3 Bucket Versioning
resource "aws_s3_bucket_versioning" "terraform_state" {
  bucket = aws_s3_bucket.terraform_state.id
  versioning_configuration { status = "Enabled"  }
}

# S3 버킷 암호화
resource "aws_s3_bucket_server_side_encryption_configuration" "terraform_state" {
  bucket = aws_s3_bucket.terraform_state.id

  rule {
    apply_server_side_encryption_by_default { sse_algorithm = "AES256" }
  }
}
# S3 버킷 퍼블릭 액세스 차단
resource "aws_s3_bucket_public_access_block" "terraform_state" {
  bucket = aws_s3_bucket.terraform_state.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
# S3 및 DynamoDB 를 활용한 상태 파일 중앙 관리
# DynamoDB 테이블 생성 (State Locking 용)
resource "aws_dynamodb_table" "terraform_state_lock" {
  name           = var.dynamodb_table_name
  billing_mode   = "PAY_PER_REQUEST"
  hash_key       = "LockID"
  attribute {
    name = "LockID"
    type = "S"
  }
  tags = {
    Name        = var.dynamodb_table_name
    Purpose     = "Terraform State Locking"
    Environment = "shared"
  }
}
```

[코드 3] S3 기반 협업 구성 코드_ 📂 **modules/terraform-backend/main.tf**

Copyright ⓒ 2025 Team_JS. All rights reserved.

[구축 결과]



[그림 4] 버킷 구축 결과

Copyright ⓒ 2025 Team_JS. All rights reserved.

3.2. 네트워크 인프라 구성

3.2.1. Region 생성

[설계 의도 요약]

인프라 배포 시 리전 설정 혼선으로 인한 오류를 방지하고자 함.

이를 위해 Provider 단계에서 리전을 서울(ap-northeast-2)로 고정하고, YAML 변수로 리전 정보를 관리하도록 구성함.

그 결과 모든 리소스가 동일한 리전을 기준으로 일관되게 배포됨.

[관련 요청사항 번호: F-01]

항목	내용
구성 대상	AWS Provider Region
주요 설정 값	ap-northeast-2
적용 범위	모든 AWS 리소스
목적/역할	리전 혼선 방지 및 일관된 배포 기준 제공

[표 5] Region 설정 정보

[상세 구현 코드]

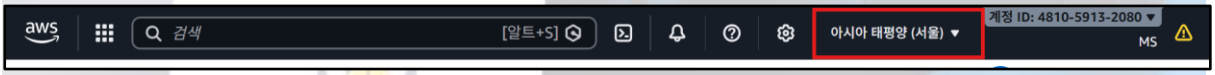
```
# ============================================================
# AWS Provider 설정
# - YAML 파일에서 정의한 리전 정보를 참조하여 Provider 구성
# ============================================================
provider "aws" {
  region = local.vars.region
}
```

[코드 4] region 설정 코드_ 📂 **seoul/main.tf**

```
# ============================================================
# Region 설정 (YAML 변수 일부 발췌)
# - Terraform Provider 및 리소스 명명에 사용
# ============================================================
region: ap-northeast-2
region_name: seoul
```

[코드 5] region 변수 관리 코드_ 📂 **seoul/variables.yml**

[구축 결과]



[그림 5] region 설정 결과

Copyright ⓒ 2025 Team_JS. All rights reserved.

3.2.2. VPC 생성

[설계 의도 요약]

클라우드 인프라의 기본 네트워크 경계를 명확히 구분하고자 함.

이를 위해 서울 리전 내에 독립적인 CIDR 대역을 가진 VPC를 생성하고, DNS 호스트네임을 활성화하도록 구성함.

그 결과 논리적으로 격리된 네트워크 환경을 확보함.

[관련 요청사항 번호: F-01]

항목	내용
구성 대상	Virtual Private Cloud (VPC)
주요 설정 값	CIDR Block: 10.2.0.0/16
적용 범위	서울 리전(ap-northeast-2) 내 네트워크
목적/역할	논리적으로 격리된 기본 네트워크 환경 구성

[표 6] VPC 네트워크 설정 정보

[상세 구현 코드]

```
# ============================================================
# VPC 리소스
# - YAML 변수로 정의된 네트워크 범위를 기반으로 VPC 생성
# ============================================================
resource "aws_vpc" "main" {
  cidr_block              = var.cidr_block
  enable_dns_hostnames    = true
  tags = {
    Name = "${var.region_name}-vpc-js"
        }
}
```

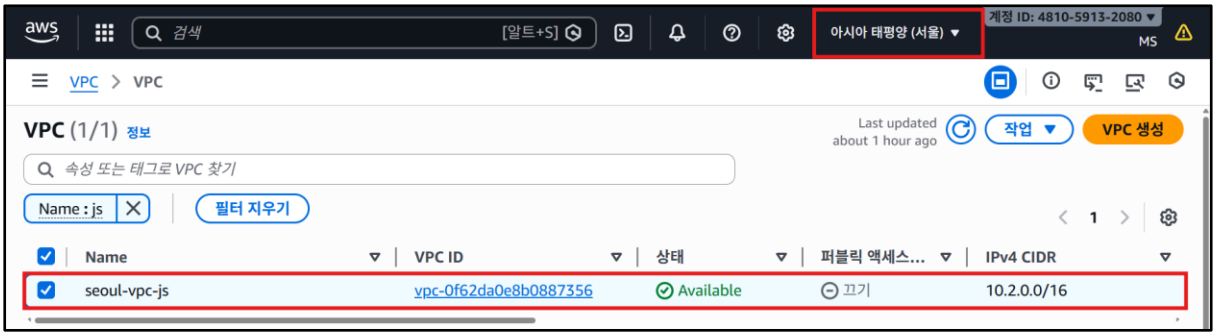
[코드 6] vpc 생성 코드_ 📂 **modules/vpc/main.tf**

```
# ============================================================
# Network 기본 설정 (YAML 변수 일부 발췌)
# - VPC 및 서브넷 구성에 사용되는 기본 네트워크 값
# ============================================================
cidr_block: "10.2.0.0/16"
az_count   :  2
```

[코드 7] vpc 변수 관리 코드_ 📂 **seoul/variables.yml**

Copyright ⓒ 2025 Team_JS. All rights reserved.

[구축 결과]



[그림 6] vpc 생성 결과

3.2.3. Subnet 생성

[설계 의도 요약]

외부에서 접근해야 하는 자원과 내부에서 보호되어야 하는 자원을 분리하고자 함.

이를 위해 Public 서브넷과 Private 서브넷을 구분하여 구성하고, Public 서브넷에는 Public IP 자동 할당을 적용하며 Private 서브넷에는 외부 직접 접근을 차단함. 또한 각 서브넷을 2 개의 가용 영역에 분산 배치하여 장애 발생 시에도 서비스가 지속되도록 구성함.

그 결과 접근 제어가 명확한 네트워크 구조를 확보하고, 고가용성을 고려한 인프라 운영이 가능함.

[관련 요청사항 번호: F-01]

항목	내용			
구성 대상	퍼블릭 서브넷 (Public)		프라이빗 서브넷 (Private)	
리소스 명	seoul-public-subnet-js-2	seoul-private-subnet-js-1	10.2.0.0/24	10.2.128.0/24
CIDR 블록	seoul-public-subnet-js-2	seoul-private-subnet-js-2	10.2.1.0/24	10.2.129.0/24
가용 영역	ap-northeast-2a	ap-northeast-2c	ap-northeast-2a	ap-northeast-2c
목적/역할	외부 접근 자원과 내부 핵심 자원 분리			

[표 7] subnet 생성 정보

Copyright ⓒ 2025 Team_JS. All rights reserved.

[상세 구현 코드]

```
# ==============================================================
# Subnet
# - 가용 영역(AZ) 분산을 통한 고가용성 확보
# - 퍼블릭 / 프라이빗 서브넷 분리로 접근 제어 강화
# ==============================================================
data "aws_availability_zones" "zones" { }
locals {
  az_list = slice(data.aws_availability_zones.zones.names, 0, var.az_count)
}
# ----------------------------
# Public Subnet
# ----------------------------
resource "aws_subnet" "publics" {
  count = var.public_subnet_count

  vpc_id = aws_vpc.main.id
  availability_zone = local.az_list[count.index % var.az_count]
  cidr_block = cidrsubnet(var.cidr_block, var.subnet_bits, count.index)

  # 퍼블릭 서브넷은 외부 통신을 위해 Public IP 자동 할당
  map_public_ip_on_launch = true

  tags = {
    Name = "${var.region_name}-public-subnet-js-${count.index + 1}"
  }
}
# ----------------------------
# Private Subnet
# ----------------------------
resource "aws_subnet" "privates" {
  count = var.private_subnet_count

  vpc_id = aws_vpc.main.id
  availability_zone  = local.az_list[count.index % var.az_count]
  cidr_block          = cidrsubnet( var.cidr_block,
                                    var.subnet_bits,
                                    pow(2, var.subnet_bits - 1) + count.index
                                    )
  # 프라이빗 서브넷은 외부 직접 접근 차단
  map_public_ip_on_launch = false

  tags = {
    Name = "${var.region_name}-private-subnet-js-${count.index + 1}"
  }
}
```

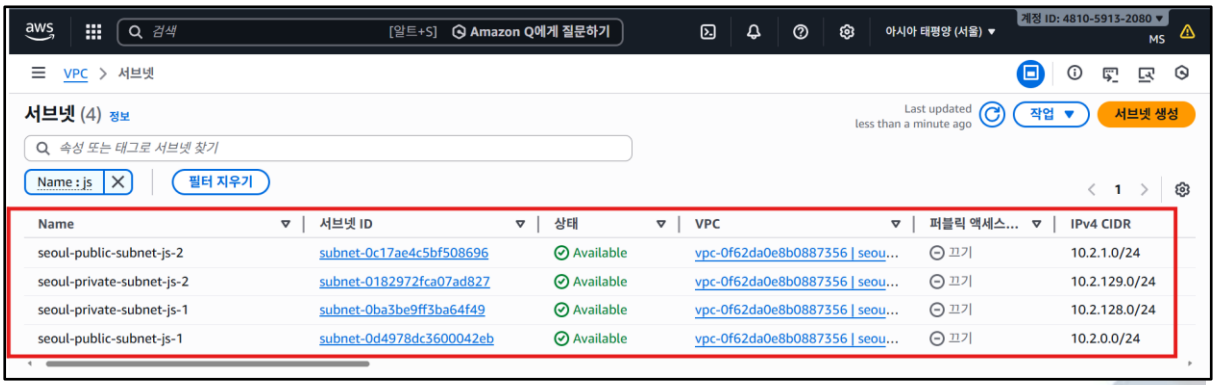
[코드 8] subnet 설정 코드_ 📂 **modules/vpc/main.tf**

Copyright ⓒ 2025 Team_JS. All rights reserved.

```yaml
# ============================================================
# Network 기본 설정
# ============================================================
cidr_block: "10.2.0.0/16"
az_count: 2
# ============================================================
# Subnet 설정
# ============================================================
public_subnet_count: 2
private_subnet_count: 2
subnet_bits: 8
```

[코드 9] subnet 변수 관리 코드_📂 **seoul/variables.yml**

[구축 결과]



[그림 7] subnet 생성 결과

Copyright ⓒ 2025 Team_JS. All rights reserved.

3.3. 인터넷 및 라우팅 구성

3.3.1. 라우팅 테이블

[설계 의도 요약]

서브넷별 역할에 따라 네트워크 트래픽 흐름을 명확히 제어하고자 함.

이를 위해 Public 서브넷과 Private 서브넷에 각각 전용 라우팅 테이블을 적용하고, Public 서브넷에는 인터넷 게이트웨이를 통한 외부 통신 경로를 설정하며 Private 서브넷에는 내부 통신만 허용하도록 구성함.

그 결과 불필요한 외부 노출을 방지하고, 서브넷 용도에 맞는 네트워크 통신 제어가 가능함.

[관련 요청사항 번호: F-02]

항목	상세 내용			
구성 대상	퍼블릭 라우팅 테이블		프라이빗 라우팅 테이블	
리소스 명	seoul-route-public		seoul-route-pri-vate-js-1	seoul-route-pri-vate-js-1
연결 서브넷	seoul-public-subnet-js-1	seoul-public-subnet-js-2	seoul-private-subnet-js-1	seoul-private-subnet-js-2
라우팅 경로 1	10.2.0.0/16 -> local		10.2.0.0/16 -> local	
라우팅 경로 2	0.0.0.0/0 -> seoul-igw-js		-	
목적/역할	IGW 연결을 통해 외부와 통신		서브넷별 테이블 분리 및 내부 통신	

[표 8] 라우팅 테이블 설정 정보

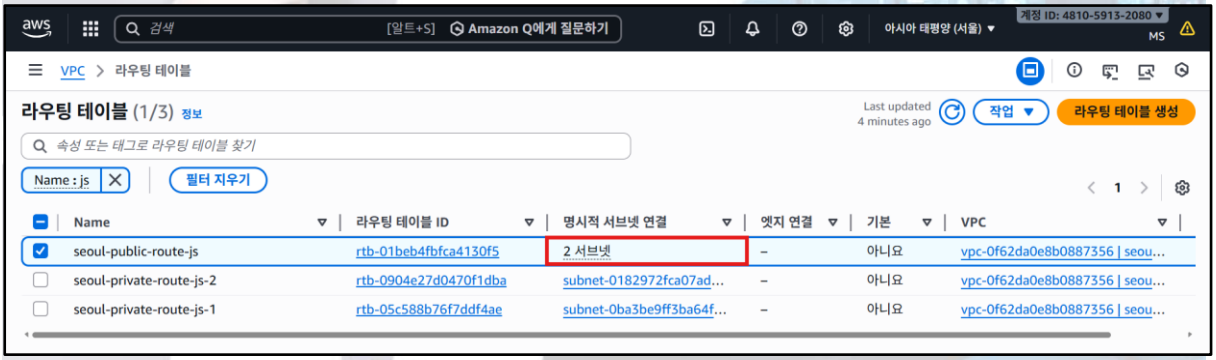
Copyright ⓒ 2025 Team_JS. All rights reserved.

[상세 구현 코드]

```tf
# ============================================================
# Route Table
# ============================================================
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }

  tags = {
    Name = "${var.region_name}-public-route-js"
  }
}

resource "aws_route_table" "privates" {
  count = var.private_subnet_count
  vpc_id = aws_vpc.main.id
  tags = {
    Name = "${var.region_name}-private-route-js-${count.index + 1}"
  }
}
# 서브넷 ↔ 라우팅 테이블 연결
resource "aws_route_table_association" "public" {
  count    = var.public_subnet_count
  route_table_id = aws_route_table.public.id
  subnet_id      = aws_subnet.publics[count.index].id
}

resource "aws_route_table_association" "privates" {
  count          = var.private_subnet_count
  route_table_id = aws_route_table.privates[count.index].id
  subnet_id      = aws_subnet.privates[count.index].id
}
```

[코드 10] 라우팅 테이블 설정 코드_📂 **modules/vpc/main.tf**

[구축 결과]



[그림 8] 라우팅 테이블 생성 결과

Copyright ⓒ 2025 Team_JS. All rights reserved.

3.3.2. Internet Gateway

[설계 의도 요약]

Public 서브넷에 배치된 자원이 외부 인터넷과 통신할 수 있도록 VPC 의 외부 연결 지점을 구성하고자 함.

이를 위해 VPC 에 Internet Gateway 를 생성하여 연결하고, Public 라우팅 테이블에서 해당 게이트웨이를 외부 트래픽 경로로 사용하도록 구성함.

그 결과 Public 서브넷에 위치한 자원은 외부와 양방향 통신이 가능하며, Private 서브넷은 외부로부터 직접 접근되지 않는 네트워크 구조를 유지함.

[관련 요청사항 번호: F-02]

항목	내용
구성 대상	Internet Gateways
리소스 명	seoul-igw-js
연결 대상	seoul-vpc-js
목적/역할	퍼블릭 서브넷의 외부 인터넷 접근 권한 부여

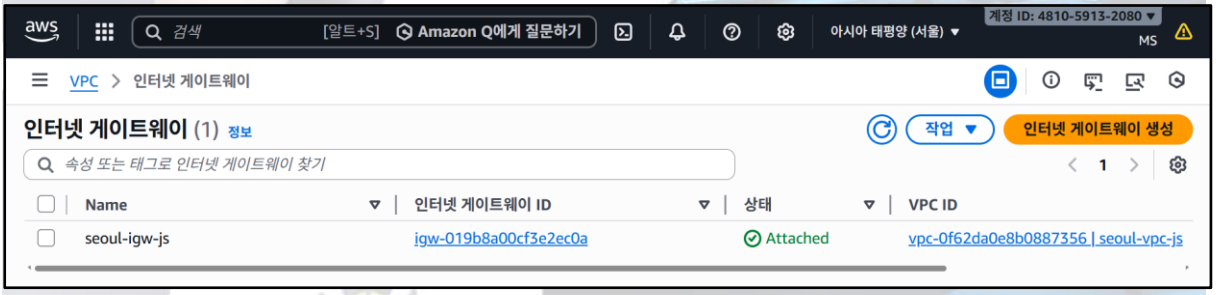
[표 9] Internet Gateway 설정 정보

[상세 구현 코드]

```
# ============================================================
# Internet Gateway
# 퍼블릭 서브넷의 외부 인터넷 통신을 위한 VPC 관문
# ============================================================
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id
  tags = {
    Name = "${var.region_name}-igw-js"
  }
}
```

[코드 11] Internet Gateway 설정 코드_📂 **modules/vpc/main.tf**

[구축 결과]



[그림 9] Internet Gateway 생성 결과

Copyright ⓒ 2025 Team_JS. All rights reserved.

3.3.3. Route53

[설계 의도 요약]

사용자가 IP 주소가 아닌 도메인 이름을 통해 서비스에 접근할 수 있도록 하고자 함.
이를 위해 VPC 구성 단계에서 퍼블릭 및 프라이빗 Hosted Zone 을 함께 생성하고, 웹 서비스와 데이터베이스 자원에 각각 역할에 맞는 도메인 레코드를 설정함.
그 결과 외부 사용자는 웹 서비스를 도메인으로 접근할 수 있으며, 내부 자원 간 통신은 프라이빗 도메인을 통해 관리되어 접근 범위가 명확히 구분됨.

[관련 요청사항 번호: F-03]

항목	상세 내용	
구성 대상	Wordpress	RDS
Zone 타입	Public	Private
도메인 명	www.js.it-edu.org	database.js.it-edu.org
레코드 타입	CNAME	
목적/역할	외부 서비스 접근 도메인 제공 및 내부 자원 통신 식별	

[표 10] Route53 설정 정보

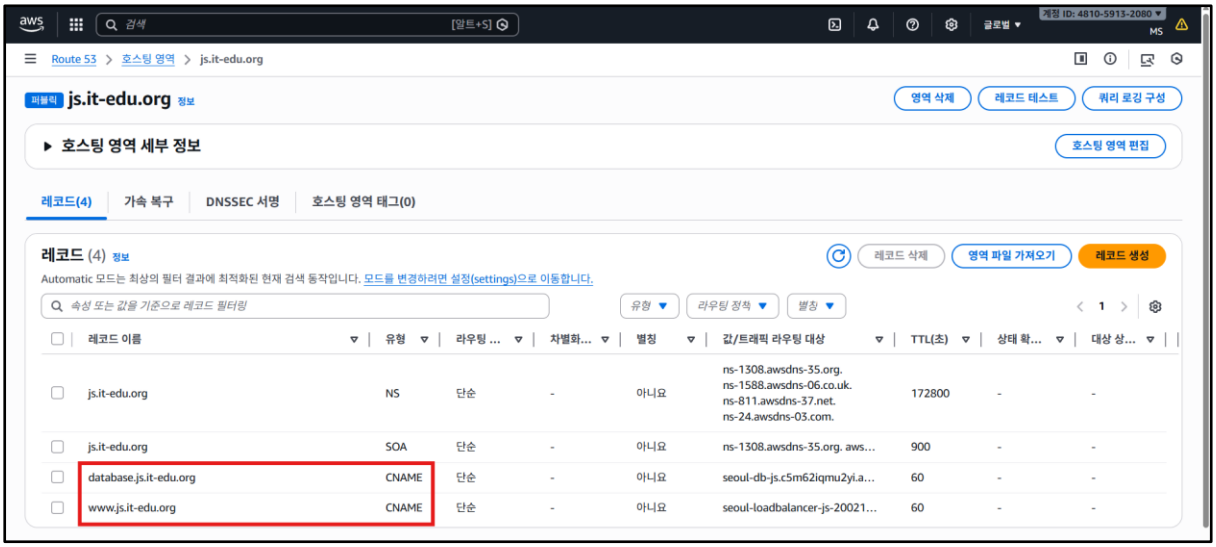
[상세 구현 코드]

```
# ============================================================
# Route53
# 서비스 접근성을 위한 도메인 관리
# ============================================================
resource "aws_route53_record" "records" {
  for_each = var.records
  zone_id  = each.value.zone_id
  name     = each.value.name
  type     = "CNAME"
  records  =
       [each.value.target_type == "rds" ? var.rds_endpoint : var.lb_dns_name]
  ttl = each.value.ttl
}
```

[코드 12] Route53 설정 코드_ 📂 **modules/route53/main.tf**

Copyright ⓒ 2025 Team_JS. All rights reserved.

[구축 결과]



[그림 10] Route53 생성 결과

Copyright ⓒ 2025 Team_JS. All rights reserved.

3.4. EC2 및 웹 서비스 구성

3.4.1. EC2

[설계 의도 요약]

웹 서비스를 실행할 컴퓨팅 자원을 표준화된 구성으로 배치하고자 함.

이를 위해 AMI 와 인스턴스 타입을 변수로 관리하고, public 값에 따라 Public/Private 서브넷을 자동 선택하도록 구성함.

또한 for_each 기반으로 다중 인스턴스를 생성하도록 구성하여 가용 영역 분산 배치가 가능하도록 함.

그 결과 동일한 기준의 서버를 반복적으로 생성·변경할 수 있으며, 배치 위치를 분리하여 접근 제어가 가능함.

[관련 요청사항 번호: F-05]

항목	내용
구성 대상	EC2 Instances
인스턴스 명	Seoul-ec2-js-{each.key}
AMI ID	Amazon Linux 2023 AMI
인스턴스 타입	t3.micro
배치 서브넷	public 값에 따라 Public/Private Subnet 자동 할당
보안 그룹	sg_name 기준으로 Security Group ID 매핑 적용
초기 설정	user_data 변수 전달
목적/역할	역할별 서버 구성을 템플릿화하여 반복 생성 및 변경 가능

[표 11] ec2 설정 정보

Copyright ⓒ 2025 Team_JS. All rights reserved.

[상세 구현 코드]

```
# ============================================================
# EC2 Instance Map
# 인스턴스 설정을 평면화하여 for_each 기반 동적 생성에 사용
# ============================================================
locals {
  # 인스턴스 유형별 설정을 개별 EC2 단위로 분해
  ec2_map = merge([
    for k, v in var.ec2-instances : {
      for i in range(v.count) : "${k}-${i + 1}" => merge(v, {
       # public 여부에 따라 퍼블릭 / 프라이빗 서브넷 자동 할당
        subnet_id = v.public
                ? var.public_subnet_ids[i % length(var.public_subnet_ids)]
                : var.private_subnet_ids[i % length(var.private_subnet_ids)]
      })
    }
  ]...)
}

# ============================================================
# EC2 Instance
# for_each 를 활용한 다중 EC2 인스턴스 생성
# ============================================================
resource "aws_instance" "ec2s" {
  for_each = local.ec2_map

  ami = each.value.ami
  instance_type = each.value.instance_type
  subnet_id = each.value.subnet_id

  # 보안 그룹
  security_groups = [var.sg_ids[each.value.sg_name]]
  # SSH 키 페어
  key_name = var.key_name
  # 초기 설정 스크립트
  user_data = each.value.user_data

  tags = {
      Name = "${var.region_name}-ec2-js-${each.key}"
  }
}
```

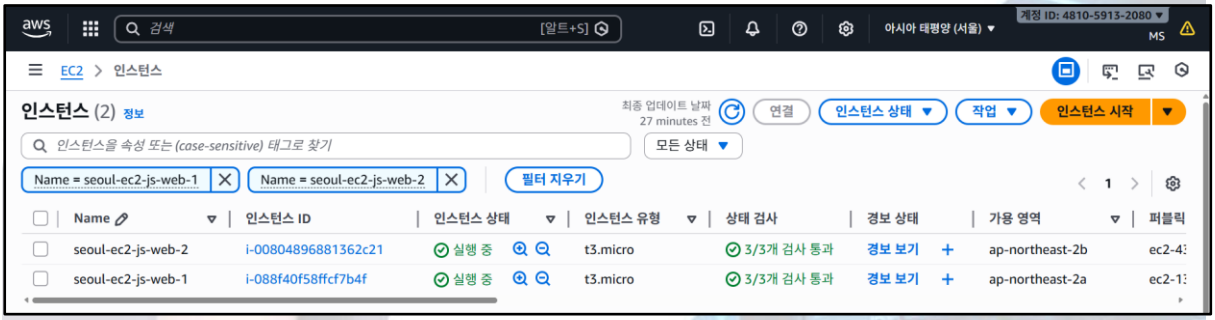
[코드 13] ec2 생성 코드_ 📂 **modules/ec2/main.tf**

Copyright ⓒ 2025 Team_JS. All rights reserved.

```yaml
# EC2 인스턴스 정의 (count 가 0이면 생성되지 않음)
ec2_instances:
  # 웹 서버 인스턴스
  web:
    count: 0
    ami:  ami-07b70db997adb22fb
    instance_type:  t3.micro
    sg_name: web
    public: true
  # 데이터베이스 서버 인스턴스
  db:
    count: 0
    ami:  ami-07b70db997adb22fb
    instance_type:  t3.micro
    sg_name: db
    public: false
    user_data:
  # DNS 서버 인스턴스
  dns:
    count: 0
    ami:  ami-07b70db997adb22fb
    instance_type:  t3.micro
    sg_name: dns
    public: true
    user_data:
```

[코드 14] ec2 변수 관리 코드_ 📂 **seoul/variables.yml**

[구축 결과]



[그림 11] EC2 생성 결과

Copyright ⓒ 2025 Team_JS. All rights reserved.

3.5. AMI 기반 표준 이미지 생성

3.5.1. 웹서비스 생성 및 AMI 이미지 생성 및 연결

[Step 1] EC2 인스턴스 생성 및 접속 확인

Terraform 자동화 코드를 통해 EC2 인스턴스를 생성한 후, Linux 환경에 접속하여 인스턴스가 정상적으로 생성 및 동작 중임을 확인함.

[Step 2] http 설치

웹 서비스 구동을 위해 httpd를 설치함.

```
[root@ip-10-2-0-191 ~]# dnf install -y httpd
systemctl enable --now httpd



Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
================================================================================================
 Package                                    Architecture                Version
================================================================================================
Installing:
 httpd                                      x86_64                      2.4.65-1.amzn2023.0.2
Installing dependencies:
 apr                                        x86_64                      1.7.5-1.amzn2023.0.4
```

[그림 12] http 설치

[Step 3] WordPress 구동을 위한 패키지/의존성 설치

WordPress 실행에 필요한 추가 패키지 및 의존성을 설치하여 서비스 구동 환경을 구성함.

```
[root@ip-10-2-0-191 tmp]# sudo dnf install -y php php-mysqlnd php-gd php-curl php-mbstring php-xml php-zip
Last metadata expiration check: 0:24:18 ago on Tue Dec 30 05:22:57 2025.
Dependencies resolved.
================================================================================================
 Package                                    Architecture                Version
================================================================================================
Installing:
 php8.4                                     x86_64                      8.4.14-1.amzn2023.0.1
 php8.4-gd                                  x86_64                      8.4.14-1.amzn2023.0.1
 php8.4-mbstring                            x86_64                      8.4.14-1.amzn2023.0.1
 php8.4-mysqlnd                             x86_64                      8.4.14-1.amzn2023.0.1
 php8.4-zip                                 x86_64                      8.4.14-1.amzn2023.0.1
```

[그림 13] 패키지 설치

Copyright ⓒ 2025 Team_JS. All rights reserved.

[Step 4] WordPress 다운로드 및 초기 설정

WordPress 설치 파일을 다운로드하고 설정 파일을 구성함. wp-config.php 생성 및 설정을 위해 웹 서버 계정(apache)이 파일을 생성·수정할 수 있도록 소유자를 조정함. 또한 웹 서버를 기동하고 WordPress 파일을 배포 경로로 이동함.

```
[root@ip-10-2-0-191 ~]# cd /tmp
wget https://wordpress.org/latest.tar.gz
tar -xzf latest.tar.gz
--2025-12-30 05:24:55--  https://wordpress.org/latest.tar.gz
Resolving wordpress.org (wordpress.org)... 198.143.164.252, 2607:f978:5:8002::c68f:a4fc
Connecting to wordpress.org (wordpress.org)|198.143.164.252|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 27058021 (26M) [application/octet-stream]
Saving to: 'latest.tar.gz'

latest.tar.gz                       100%[=================================================
```

[그림 14] tar 파일 해제

```
[root@ip-10-2-0-191 tmp]# sudo mv wordpress /var/www/html/
[root@ip-10-2-0-191 tmp]# sudo chown -R apache:apache /var/www/html/wordpress
sudo chmod -R 755 /var/www/html/wordpress
[root@ip-10-2-0-191 tmp]# sudo systemctl restart httpd
[root@ip-10-2-0-191 tmp]#
```

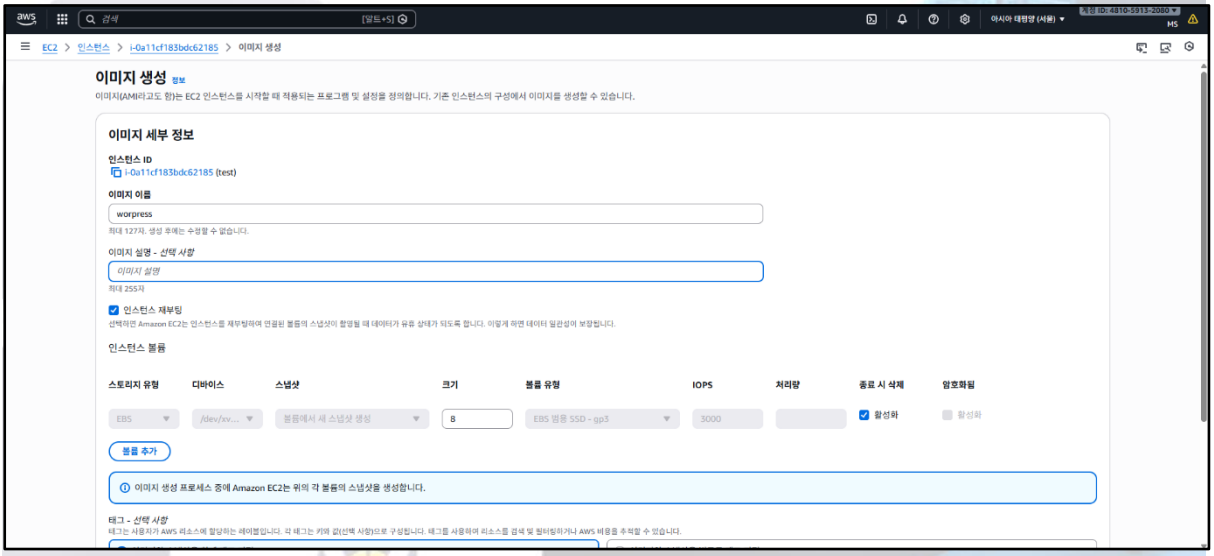
[그림 15] 소유자 apache 변경 및 httpd start

```
[root@ip-10-2-0-191 tmp]# mv /var/www/html/wordpress/* /var/www/html/
[root@ip-10-2-0-191 tmp]#
```

[그림 16] wordpress 하위 파일 html로 이동

[Step 5] AMI 이미지 생성

구축 완료된 인스턴스를 기반으로 AMI 이미지를 생성하여 동일한 구성의 서버를 재사용할 수 있도록 함.



[그림 17] 이미지 생성 과정

Copyright ⓒ 2025 Team_JS. All rights reserved.

[Step 6] AMI 생성 결과 확인

생성된 AMI가 정상적으로 등록되었는지 확인함.

[그림 18] 이미지 생성 확인

3.5.2. Launch Template 구성

[설계 의도 요약]

Auto Scaling 및 인스턴스 생성 시 적용할 EC2 시작 설정을 표준화하기 위해 Launch Template 를 구성하였다.

AMI, 인스턴스 타입, 네트워크 및 보안 설정을 템플릿으로 정의하여, 인스턴스 생성 시 동일한 기준이 자동으로 적용되도록 하였다.

이를 통해 서버 생성 방식의 일관성을 확보하고, 확장 및 변경 시 설정 오류를 최소화하도록 구성하였다.

[관련 요청사항 번호: F-05]

항목	내용
구성 대상	Launch Template
템플릿 명	seoul-template-js
적용 대상	EC2 Instance (Auto Scaling Group 연계)
기준 AMI	project-mega (Amazon Linux 2023 AMI)
기본 인스턴스 타입	t3.micro
네트워크 설정	public 값에 따라 Public / Private Subnet 자동 적용
보안 설정	sg_name 기준으로 Security Group ID 매핑
초기 설정 정의	user_data 스크립트를 템플릿에 포함
생성 방식	for_each 기반 다중 템플릿 정의
목적 / 역할	EC2 인스턴스 생성 기준을 표준화하여 확장 및 변경 시 동일한 설정이 자동 적용되도록 함

[표 12] Template 설정 정보

Copyright ⓒ 2025 Team_JS. All rights reserved.

[상세 구현 코드]

```
# ============================================================
# Launch Template 모듈 - EC2 시작 템플릿 생성
# ============================================================
# User Data 스크립트 생성 (로컬 변수)
locals {
  user_data_scripts = {
    for k, v in var.templates : k => (
      v.db_config_name != "" && contains(keys(var.db_config), v.db_config_name)
      ? <<-EOF
#!/bin/bash
# WordPress DB 연결 설정 스크립트 (자동 생성)
# 템플릿: ${v.name}
# DB 설정: ${v.db_config_name}

sed -i "s/<DB_NAME>/${var.db_config[v.db_config_name].db_name}/" /var/www/html/wp-
config.php
sed -i "s/<DB_USER>/${var.db_config[v.db_config_name].db_user}/" /var/www/html/wp-
config.php
sed -i "s/<DB_PASSWORD>/${var.db_config[v.db_config_name].db_password}/"
/var/www/html/wp-config.php
sed -i "s/<DB_HOST>/${var.db_config[v.db_config_name].db_host}/" /var/www/html/wp-
config.php
EOF
      : ""  )}}
# EC2 시작 템플릿 생성
resource "aws_launch_template" "templates" {

  for_each = var.templates
  name = each.value.name
  description = each.value.description
  image_id = var.ami_images[each.value.ami_name]
  key_name = var.key_name

  # IAM 인스턴스 프로필 (SSM 접근용, 선택사항)
  dynamic "iam_instance_profile" {
    for_each = var.iam_instance_profile_name != "" ? [1] : []
    content {
      name = var.iam_instance_profile_name
    }
  }

  network_interfaces {
    associate_public_ip_address = each.value.public
    security_groups = [var.sg_ids[each.value.sg_name]]
  }
  user_data = base64encode(local.user_data_scripts[each.key])

  tags = {
      Name = "${var.region_name}-ec2-js-${each.key}"
  }}
```

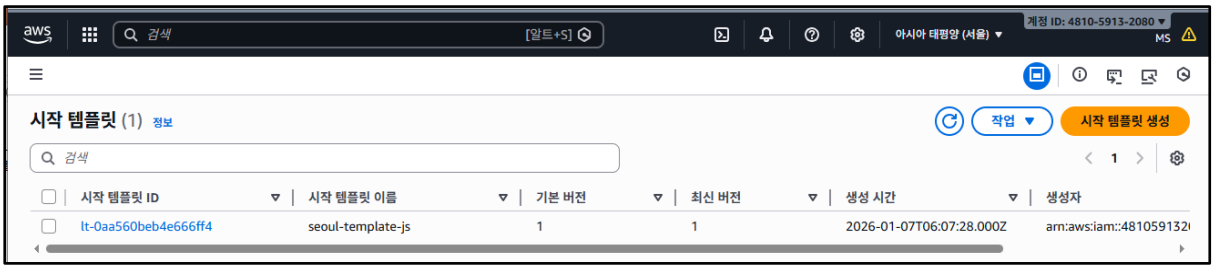
[코드 15] Template 생성 코드_ 📂 **modules/launch_template/main.tf**

Copyright ⓒ 2025 Team_JS. All rights reserved.

```yaml
templates:
  wp-dynamic:
    name: "seoul-template-js"
    description: "template for auto-scailing-dynamic"
    ami_name: project-mega
    public: true
    sg_name: web
    db_config_name: wordpress
    user_data:
```

[코드 16] Template 변수 관리 코드_ 📂 **seoul/variables.yml**

[구축 결과]



[그림 19] Template 생성 결과

Copyright ⓒ 2025 Team_JS. All rights reserved.

3.6. 데이터베이스 구성

3.6.1. RDS 생성

[설계 의도 요약]

애플리케이션 데이터를 안정적으로 저장하고 관리할 데이터베이스 환경을 구성하고자 함.

이를 위해 RDS 인스턴스를 Private Subnet 에 배치하고, 전용 DB 서브넷 그룹을 구성하여 외부 인터넷으로부터의 직접 접근을 차단함.

또한 YAML 변수로 DB 엔진, 버전, 인스턴스 사양을 관리하고 스냅샷 기반 복원 방식을 적용함.

그 결과 보안이 강화된 데이터베이스 환경을 구축하고, 설정 변경 및 복원이 유연한 DB 운영이 가능함.

[관련 요청사항 번호: F-04]

항목	내용
구성 대상	RDS Databases
DB 엔진 및 버전	MariaDB 11.4.5
DB 식별자	seoul-db-js
인스턴스 타입	db.t4g.micro
스토리지 설정	gp3, 20GB
DB 서브넷 그룹	mysql-subnet-group (프라이빗 서브넷 2개 지정)
보안 그룹	seoul-sg-js-rds (3306 포트 허용)
퍼블릭 접근	비활성화 (False)
백업 및 복원	wordpress-db 스냅샷 활용
목적/역할	프라이빗 서브넷 배포 및 웹 서버 전용 접속 허용

[표 13] RDS 설정 정보

Copyright ⓒ 2025 Team_JS. All rights reserved.

[상세 구현 코드]

```
# ============================================================
# RDS Snapshot Data Source
# ============================================================
data "aws_db_snapshot" "latest_snapshot" {
  count                   = var.db_instance.latest_snapshot && var.db_in-
stance.snapshot_identifier != "" ? 1 : 0
  db_snapshot_identifier = var.db_instance.snapshot_identifier
}
# DB Subnet Group
resource "aws_db_subnet_group" "db" {
  name        = var.rds_subnet_group.name
  description = var.rds_subnet_group.description
  subnet_ids  = var.subnet_ids
}
# RDS Instance
resource "aws_db_instance" "db-instance" {
  depends_on     = [aws_db_subnet_group.db]
  # DB 엔진 설정
  engine         = var.db_instance.engine
  engine_version = var.db_instance.engine_version
  identifier     = var.db_instance.identifier
  # 인증 정보
  username = var.db_instance.username
  password = var.db_instance.password
  # 인스턴스 사양
  instance_class    = var.db_instance.instance_class
  storage_type      = var.db_instance.storage_type
  allocated_storage = var.db_instance.allocated_storage
  multi_az          = var.db_instance.multi_az
  # 네트워크 설정
  db_subnet_group_name   = aws_db_subnet_group.db.name
  vpc_security_group_ids = [for name in var.db_instance.sg_names :
var.sg_ids[name]]
  publicly_accessible    = var.db_instance.publicly_accessible
  # 스냅샷이 있을 때만 snapshot_identifier 사용
  snapshot_identifier = var.db_instance.latest_snapshot && var.db_instance.snap-
shot_identifier != "" ? data.aws_db_snapshot.latest_snapshot[0].id : (var.db_in-
stance.snapshot_identifier != "" ? var.db_instance.snapshot_identifier : null)
  # 종료 시 스냅샷 설정
  skip_final_snapshot       = true
  deletion_protection       = false
  final_snapshot_identifier = "${var.db_instance.identifier}-${re-
place(timestamp(), ":", "-")}"
}
```

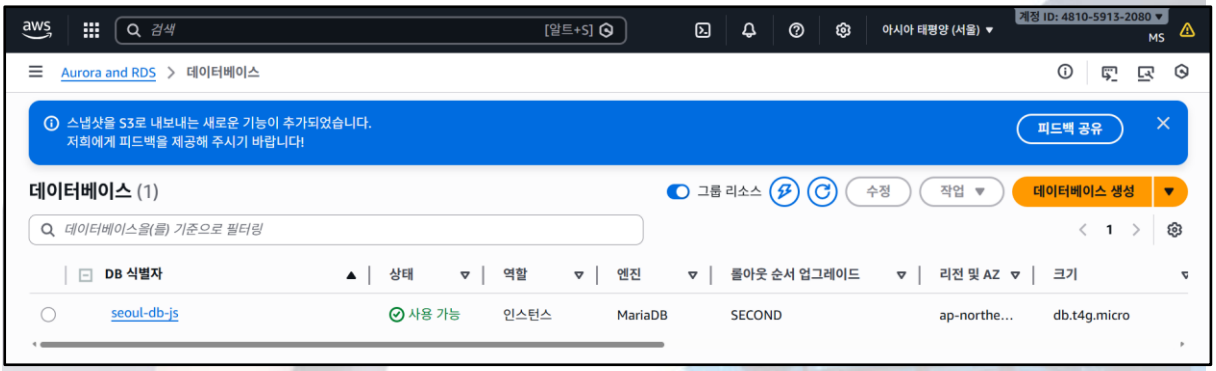
[코드 17] RDS 생성 코드_ 📂 **modules/rds/main.tf**

Copyright ⓒ 2025 Team_JS. All rights reserved.

```yaml
# RDS 서브넷 그룹 설정
rds_subnet_group:
  name : "seoul-dbinstance-js"
  description : "db_subnet group for mysql-cluster"
# RDS 인스턴스 설정
db_instance :
  engine              : "mariadb"
  engine_version      : "11.4.5"
  identifier          : "seoul-db-js"
  # 스냅샷 복원 설정
  snapshot_identifier : "wordpress-db"
  latest_snapshot     : false
  # DB 인증 정보
  username            : "root"
  password            : "Passw0rd"
  # 인스턴스 사양
  instance_class      : "db.t4g.micro"
  storage_type        : "gp3"
  allocated_storage   : 20
  multi_az            : false
  # 네트워크 설정
  publicly_accessible : false
  sg_names            : ["rds"]
  source_db_identifer : "wordpress-setup"
```

[코드 18] RDS 변수 관리 코드_📂 **seoul/variables.yml**

[구축 결과]



[그림 20] RDS 생성 결과

Copyright ⓒ 2025 Team_JS. All rights reserved.

3.6.2. 데이터베이스 생성 후 RDS 스냅샷 생성 및 연결

[Step 1] MariaDB 클라이언트 설치

RDS MariaDB 인스턴스에 접속하기 위해 EC2 인스턴스에 MariaDB를 설치함.

```
[root@ip-10-2-0-191 tmp]# dnf install -y mariadb1011
Last metadata expiration check: 0:11:55 ago on Tue Dec 30 05:22:57 2025.
Dependencies resolved.
================================================================================================
 Package                                    Architecture            Version
================================================================================================
Installing:
 mariadb1011                                x86_64                  3:10.11.13-1.amzn2023.0.1
Installing dependencies:
 mariadb-connector-c                        x86_64                  3.3.10-1.amzn2023.0.1
 mariadb-connector-c-config                 noarch                  3.3.10-1.amzn2023.0.1
 mariadb1011-common                         x86_64                  3:10.11.13-1.amzn2023.0.1
 perl-B                                     x86_64                  1.80-477.amzn2023.0.7
 perl-DBI                                   x86_64                  1.643-7.amzn2023.0.3
 perl-Data-Dumper                           x86_64                  2.174-460.amzn2023.0.2
 perl-FileHandle                            noarch                  2.03-477.amzn2023.0.7
 perl-Math-BigInt                           noarch                  1:1.9998.39-2.amzn2023.0.2
 perl-Math-BigRat                           noarch                  0.2624-500.amzn2023.0.2
```

[그림 21] Mariadb 설치

[Step 2] 데이터베이스 초기 설정

MariaDB 클라이언트를 통해 RDS에 접속한 후, WordPress 연동을 위한 데이터베이스 및 사용자 계정을 초기 설정함.

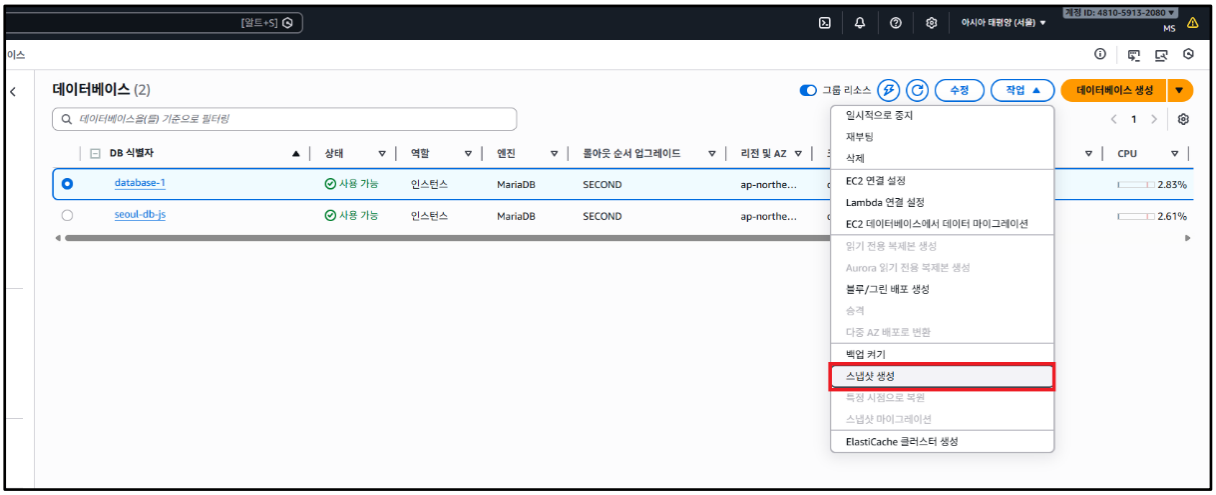
```
[root@ip-10-2-0-191 tmp]# mysql -u root -h database-1.c5m62iqmu2y1.ap-northeast-2.rds.amazonaws.com -pPassw0rd
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 18
Server version: 11.4.8-MariaDB managed by https://aws.amazon.com/rds/

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE wpDB DEFAULT CHARACTER SET utf8;
Query OK, 1 row affected (0.002 sec)

MariaDB [(none)]> CREATE USER 'wpadmin'@'%' IDENTIFIED BY 'P@ssw0rd';
Query OK, 0 rows affected (0.003 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON wpDB.* TO 'wpadmin'@'%';
Query OK, 0 rows affected (0.002 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.004 sec)

MariaDB [(none)]> EXIT;
Bye
```

[그림 22] 데이터베이스 초기 생성 과정

Copyright ⓒ 2025 Team_JS. All rights reserved.

[Step 3] 데이터베이스 스냅샷 생성

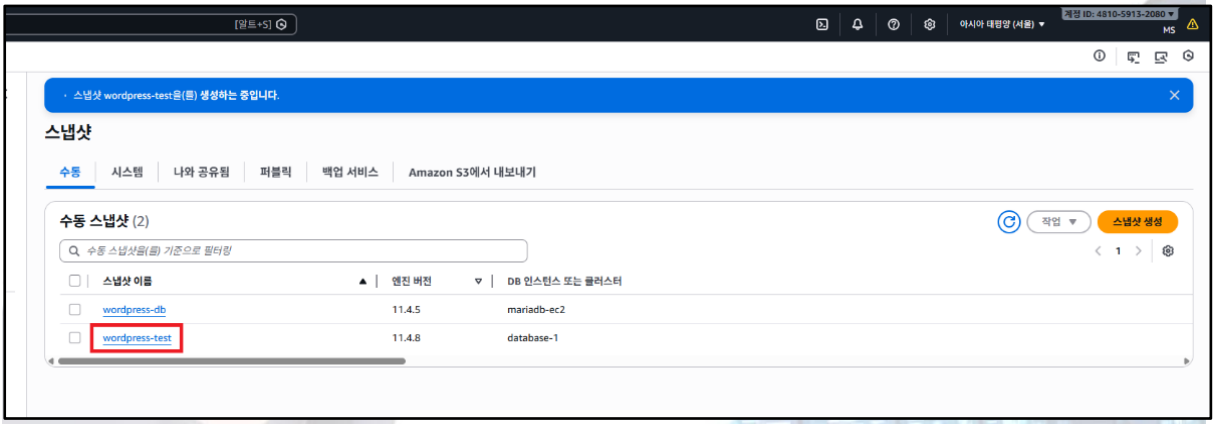
초기 설정이 완료된 데이터베이스 상태를 기준으로 스냅샷을 생성하여, 향후 복원 및 재배포에 활용할 수 있도록 구성함.



[그림 23] 스냅샷 생성

[Step 4] 스냅샷 생성 완료

생성된 데이터베이스 스냅샷이 정상적으로 등록되었음을 확인함



[그림 24] 스냅샷 생성 완료

Copyright ⓒ 2025 Team_JS. All rights reserved.

3.7. 고가용성 구성

3.7.1. Target Group

[설계 의도 요약]

로드밸런서가 트래픽을 전달할 백엔드 서버 집합을 명확히 정의하고자 함.

이를 위해 웹 서비스를 수행하는 EC2 인스턴스를 Target Group 으로 구성하고, 헬스 체크를 통해 정상 동작 여부를 지속적으로 확인하도록 설정함.

또한 비정상 상태의 인스턴스는 자동으로 트래픽 대상에서 제외되도록 구성함.

그 결과 장애 인스턴스로의 트래픽 전달을 방지하고, 안정적인 서비스 운영이 가능함.

[관련 요청사항 번호: F-05]

항목	내용
구성 대상	Target Groups
대상 그룹 명	seoul-targetgroup-js
대상 유형	Instance
프로토콜 및 포트	HTTP / 80
적용 VPC	seoul-vpc-js
헬스 체크	/index.php
목적/역할	웹 서버 인스턴스를 그룹화하여 ALB와 연동

[표 14] TG 설정 정보

Copyright ⓒ 2025 Team_JS. All rights reserved.

[상세 구현 코드]

```
# ============================================================
# 타겟 그룹 생성
# 로드 밸런서(ALB/NLB)가 트래픽을 전달할 대상 리소스 정의
# ============================================================
resource "aws_lb_target_group" "name" {
  target_type         = var.target_group.target_type
  name                = var.target_group.name
  protocol            = var.target_group.protocol
  port                = var.target_group.port
  ip_address_type     = var.target_group.ip_address_type
  vpc_id              = var.vpc_id
  protocol_version    = var.target_group.protocol_version

  # ============================================================
  # 헬스 체크 설정
  # ============================================================
  health_check {
    protocol            = var.target_group.health_check.protocol
    path                = var.target_group.health_check.path
    port                = var.target_group.health_check.port
    healthy_threshold   = var.target_group.health_check.healthy_threshold
    unhealthy_threshold = var.target_group.health_check.unhealthy_threshold
    timeout             = var.target_group.health_check.timeout
    interval            = var.target_group.health_check.interval
    matcher             = var.target_group.health_check.matcher
  }
}
```

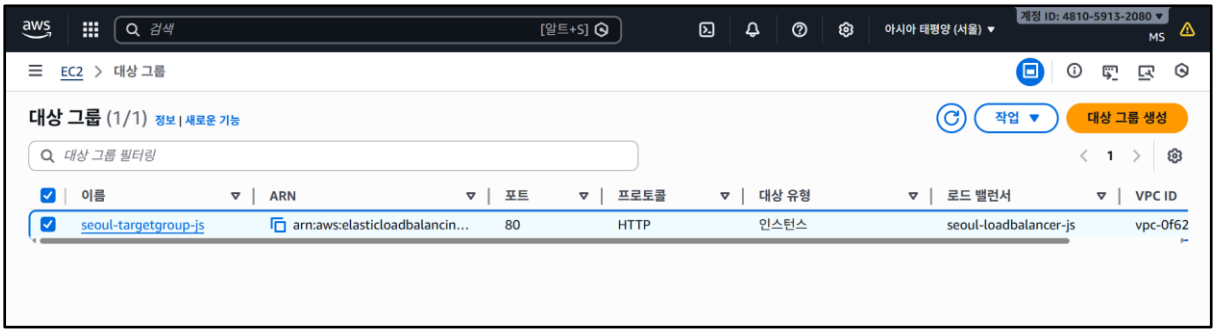
[코드 19] TG 설정 코드_ 📂 **modules/target-group/main.tf**

```
target_group:
  target_type         : instance
  name                : seoul-targetgroup-js
  protocol            : "HTTP"
  port                : 80
  ip_address_type     : "ipv4"
  protocol_version    : "HTTP1"
  # 헬스 체크 설정 (인스턴스 상태 확인)
  health_check:
    protocol            : "HTTP"
    path                : "/index.php"
    port                : "traffic-port"
    healthy_threshold   : 2
    unhealthy_threshold: 5
    timeout             : 15
    interval            : 30
    matcher             : "200-399"
```

[코드 20] TG 변수 관리 코드_ 📂 **seoul/variables.yml**

Copyright ⓒ 2025 Team_JS. All rights reserved.

[구축 결과]



[그림 25] TG 생성 결과

3.7.2. Load Balancer

[설계 의도 요약]

외부에서 유입되는 HTTP 요청을 단일 진입점에서 수신하고자 함.

이를 위해 Application Load Balancer 를 구성하여 퍼블릭 서브넷에 배치하고, 수신된 트래픽을 Target Group 에 등록된 EC2 인스턴스로 분산 전달하도록 설정함.

또한 가용 영역에 분산된 서브넷을 사용하여 특정 AZ 장애 발생 시에도 트래픽 처리가 가능하도록 구성함.

그 결과 단일 서버 장애로 인한 서비스 중단을 방지하고, 안정적인 트래픽 처리 환경을 확보함.

[관련 요청사항 번호: F-05]

항목	내용	
구성 대상	Application Load Balancer	
ALB 명	seoul-loadbalancer-js	
로드 밸런서 유형	인터넷 연결 (Internet-facing)	
IP 주소 유형	IPv4	
VPC	seoul-vpc-js	
배치 서브넷	seoul-public-subnet-js-1	seoul-public-subnet-js-1
보안 그룹	seoul-sg-lb (80 포트 허용)	
리스너 구성	HTTP(80) 요청을 Target Group으로 전달	
목적/역할	외부 트래픽 수신 및 백엔드 서버 분산 처리	

[표 15] LB 설정 정보

Copyright ⓒ 2025 Team_JS. All rights reserved.

[상세 구현 코드]

```
# ============================================================
# Application Load Balancer (ALB) 생성
# HTTP 트래픽 처리를 위한 L7 로드 밸런서
# ============================================================
resource "aws_lb" "this" {
  load_balancer_type = var.load_balancer.load_balancer_type
  name               = var.load_balancer.name
  internal           = var.load_balancer.internal
  ip_address_type    = var.load_balancer.ip_address_type
  subnets            = var.public_subnet_ids
  security_groups    = [var.sg_ids[var.load_balancer.sg_name]]
}
# ============================================================
# ALB Listener 생성
# 수신 포트/프로토콜에 따라 트래픽 처리 방식 정의
# ============================================================
resource "aws_lb_listener" "this" {
  load_balancer_arn  = aws_lb.this.arn
  port               = var.load_balancer_listener.port
  protocol           = var.load_balancer_listener.protocol

  default_action {
    type             = var.load_balancer_listener.type
    target_group_arn = var.target_group_arn
  }
```

[코드 21] LB 설정 코드_ 📂 **modules/load-balancer/main.tf**

```
load_balancer:
  load_balancer_type : application
  name               : seoul-loadbalancer-js
  internal           : false
  ip_address_type    : ipv4
  sg_name            : lb

load_balancer_listener:
  protocol     : HTTP
  port         : 80
  type         : forward
```

[코드 22] LB 변수 관리 코드_ 📂 **seoul/variables.yml**

[구축 결과]

[그림 26] LB 생성 결과

Copyright ⓒ 2025 Team_JS. All rights reserved.

3.7.3. Auto Scalling

[설계 의도 요약]

트래픽 변화에 따라 웹 서버 수를 자동으로 조정하여 서비스 안정성과 자원 효율성을 확보하고자 함.

이를 위해 Launch Template 을 기반으로 Auto Scaling Group 을 구성하고, 로드밸런서와 연동하여 정상 인스턴스만 서비스에 참여하도록 설정함.

또한 다수의 퍼블릭 서브넷에 인스턴스를 분산 배치하여 특정 가용 영역 장애 상황에서도 서비스가 지속되도록 구성함.

그 결과 트래픽 증가·감소에 자동 대응이 가능하며, 안정적인 서비스 운영과 비용 최적화가 가능함.

[관련 요청사항 번호: F-05]

항목	내용	
구성 대상	Auto Scaling Groups	
ASG 명	seoul-autoscaling-js	
시작 구성 방식	Launch Template (시작 템플릿)	
활용 AMI	project-mega	
최소/희망/최대 용량	최소 2 / 희망 2 / 최대 4	
배치 서브넷	seoul-public-subnet-js-1	seoul-public-subnet-js-2
대상 그룹 연동	seoul-target-js	
상태 확인 유형	ELB (로드밸런서 상태 확인 기반)	
목적/역할	트래픽 변동에 대응하는 유연한 인프라 확장 체계 구축	

[표 16] AS 설정 정보

Copyright ⓒ 2025 Team_JS. All rights reserved.

[상세 구현 코드]

Copyright ⓒ 2025 Team_JS. All rights reserved.

```
# ============================================================
# Auto Scaling Group (ASG)
# ============================================================
resource "aws_autoscaling_group" "main" {
  name                      = var.autoscaling_group.name
  vpc_zone_identifier       = var.public_subnet_ids
  target_group_arns         = [var.target_group_arn]
  health_check_type         = var.autoscaling_group.health_check_type
  health_check_grace_period  = var.autoscaling_group.health_check_grace_period

  min_size         = var.autoscaling_group.min_size
  desired_capacity = var.autoscaling_group.desired_capacity
  max_size         = var.autoscaling_group.max_size

# Mixed Instances Policy
  mixed_instances_policy {
    launch_template {
      launch_template_specification {
        launch_template_id   = var.lt_ids[var.autoscaling_group.launch_tem-
plate.template_name]
        version              = "$Latest"
      }
      dynamic "override" {
        for_each = var.autoscaling_group.launch_template.instance_type
        content {
          instance_type = override.key
          weighted_capacity = override.value
          dynamic "launch_template_specification" {
            for_each = try(var.autoscaling_group.launch_template.template_over-
rides[override.key] != null, false) ? [1] : []
            content {
              launch_template_id = var.lt_ids[var.autoscaling_group.launch_tem-
plate.template_overrides[override.key]]
              version            = "$Latest"
            }}}}}}}

# ============================================================
# Auto Scaling Policy
# ============================================================
resource "aws_autoscaling_policy" "name" {
  autoscaling_group_name     = aws_autoscaling_group.main.name
  policy_type               = var.autoscaling_policy.policy_type
  name                      = var.autoscaling_policy.name
  estimated_instance_warmup = var.autoscaling_policy.estimated_instance_warmup

    target_tracking_configuration {
    predefined_metric_specification {
      predefined_metric_type = var.autoscaling_policy.config.metric_type
    }
    target_value = var.autoscaling_policy.config.target_value
  }
}
```

[코드 23] AS 설정 코드_ 📂 **modules/auto-scaling/main.tf**

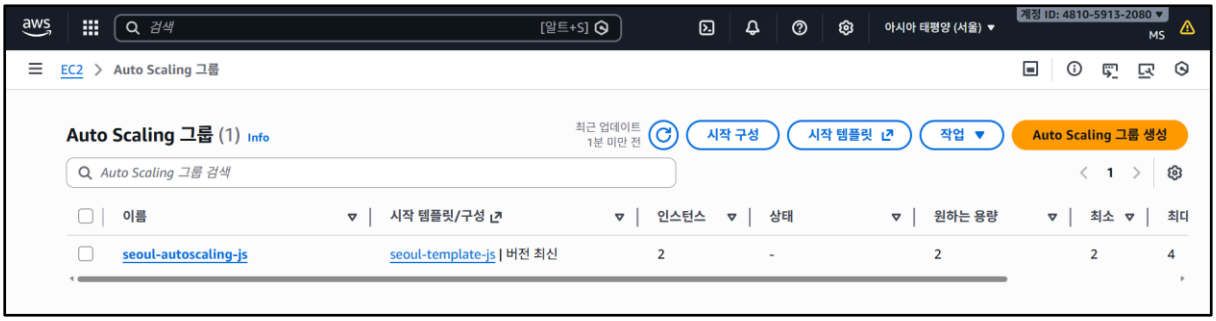
Copyright ⓒ 2025 Team_JS. All rights reserved.

```yaml
autoscaling_group:
  name                        : "seoul-autoscaling-js"
  health_check_type           : "ELB"
  health_check_grace_period   : 600
  min_size                    : 2
  desired_capacity            : 2
  max_size                    : 4
  launch_template:
    instance_type:
      t3.micro: "1"
    template_name: wp-dynamic

# ------------------------------------------------------------
# Auto Scaling Policy (스케일링 정책) 설정
# CPU 사용률 기반 자동 스케일링 규칙
# ------------------------------------------------------------
autoscaling_policy:
  policy_type                 : TargetTrackingScaling
  name                        : TargetTrackingScailing
  estimated_instance_warmup   : 60
  config:
    metric_type : ASGAverageCPUUtilization
    target_value:  60
```

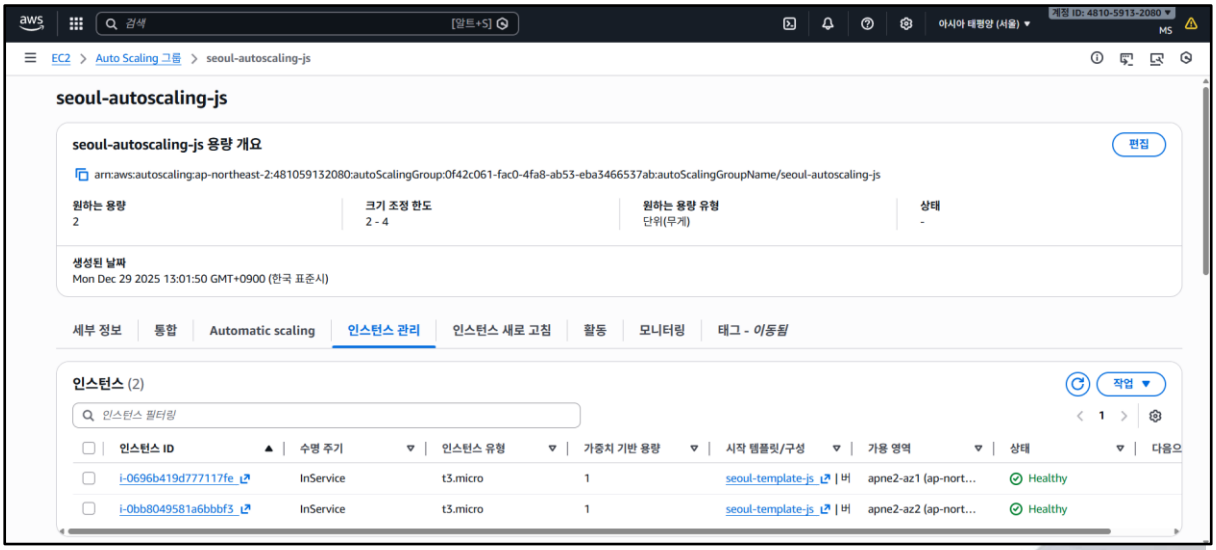
[코드 24] AS 변수 관리 코드_ 🗀 **seoul/variables.yml**

Copyright ⓒ 2025 Team_JS. All rights reserved.

[구축 결과]



[그림 27] AS 생성 결과



[그림 28] AS 생성 결과 인스턴스 관리 창

Copyright ⓒ 2025 Team_JS. All rights reserved.

3.8. 보안 기본 구성

3.8.1. Security Group 생성

[설계 의도 요약]

인프라 각 계층별로 허용된 트래픽만 통과시키는 네트워크 접근 제어 체계를 구성하고자 함.
이를 위해 웹, 로드밸런서, 데이터베이스 역할별로 보안 그룹을 분리하고, 필요한 포트와 통신 대상만 허용하도록 규칙을 정의함.
또한 최소 권한 원칙에 따라 불필요한 접근을 차단하고, 계층 간 접근 관계를 명확히 설정함.
그 결과 외부 및 내부 트래픽 흐름이 명확히 구분된 보안 구조를 확보함.

[관련 요청사항 번호: F-04, F-05]

항목	내용		
구성 대상	WEB SG	RDS SG	LB SG
Sg 명	seoul-sg-js-web	seoul-sg-js-rds	seoul-sg-js-lb
인바운드 규칙	22, 80, 443, ICMP	3306	80
아웃바운드 규칙	전체 허용	없음	내부 대역 허용
목적/역할	사용자 요청 처리	트래픽 중계	DB 접근 제어

[표 17] SG 설정 정보

Copyright ⓒ 2025 Team_JS. All rights reserved.

[상세 구현 코드]

```
# ============================================================
# Security Group
# 역할별 보안 그룹을 동적으로 생성
# ============================================================
resource "aws_security_group" "sg" {
  for_each = var.sg
  vpc_id   = var.vpc_id
  name     = "${var.region_name}-sg-js-${each.key}"

# Ingress Rules
  dynamic "ingress" {
    for_each = each.value.ingress_rules
    content {
      protocol     = ingress.value.protocol
      from_port    = ingress.value.from_port
      to_port      = ingress.value.to_port
      cidr_blocks  = ingress.value.cidr_blocks
    }
  }
  tags = {
    Name = "${var.region_name}-sg-js-${each.key}"
  }

# Egress Rules
  dynamic "egress" {
    for_each = each.value.egress_rules
    content {
      protocol    = egress.value.protocol
      from_port   = egress.value.from_port
      to_port      = egress.value.to_port
      cidr_blocks = egress.value.cidr_blocks
    }}}
```

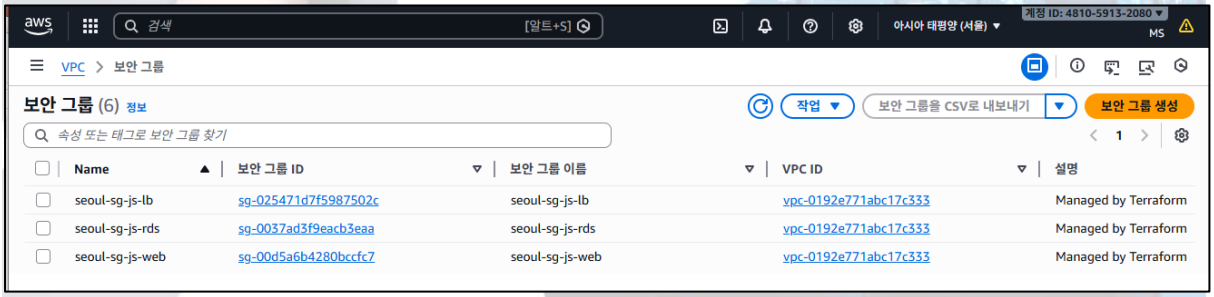
[코드 25] SG 설정 코드_ 📂 **modules/security-group/main.tf**

Copyright ⓒ 2025 Team_JS. All rights reserved.

```yaml
sg:
  web:
    ingress_rules:
      ssh:
        protocol    : tcp
        from_port   : 22
        to_port     : 22
        cidr_blocks:
          - 0.0.0.0/0
      http:
        protocol    : tcp
        from_port   : 80
        to_port     : 80
        cidr_blocks:
          - 0.0.0.0/0
      https:
        protocol    : tcp
        from_port   : 443
        to_port     : 443
        cidr_blocks:
          - 0.0.0.0/0
      icmp:
        protocol    : icmp
        from_port   : 8
        to_port     : 0
        cidr_blocks:
          - 0.0.0.0/0
    egress_rules:
      all:
        protocol    : -1
        from_port   : 0
        to_port     : 0
        cidr_blocks:
          - 0.0.0.0/0
  lb:
    ingress_rules:
      http:
        protocol    : tcp
        from_port   : 80
        to_port     : 80
        cidr_blocks:
          - 0.0.0.0/0
    egress_rules:
      http:
        protocol    : tcp
        from_port   : 80
        to_port     : 80
        cidr_blocks:
          - 10.2.0.0/16
  rds:
    ingress_rules:
      mysql:
        protocol    : tcp
        from_port   : 3306
        to_port     : 3306
        cidr_blocks:
          - 0.0.0.0/0
    egress_rules: {}
```

[코드 26] SG 변수 관리 코드_📂 **seoul/variables.yml**

[구축 결과]



[그림 29] SG 생성 결과

Copyright ⓒ 2025 Team_JS. All rights reserved.

3.8.2. 키페어 생성

[설계 의도 요약]

EC2 인스턴스에 안전하게 원격 접속할 수 있는 인증 수단을 마련하고자 함.

이를 위해 비밀번호 방식 대신 공개키 기반 인증(Key Pair)을 적용하여 인스턴스 접근 시 사용자 신원을 검증하도록 구성함.

또한 Auto Scaling 으로 생성되는 인스턴스에도 동일한 키가 적용되도록 하여 접근 방식을 일관되게 유지함.

그 결과 무차별 대입 공격 위험을 줄이고, 관리자만 접근 가능한 서버 접속 환경을 확보함.

[관련 요청사항 번호: S-03]

항목	내용
리소스 유형	EC2 Key Pair
키 알고리즘	RSA (기본 보안 표준 적용)
파일 연동	로컬 시스템의 공개키(.pub) 파일 경로 참조
적용 범위	웹 서버(EC2) 및 Auto Scaling 인스턴스 접속용
목적/역할	비밀번호 없는 안전한 서버 접속

[표 18] 키페어 설정 정보

Copyright ⓒ 2025 Team_JS. All rights reserved.

[상세 구현 코드]

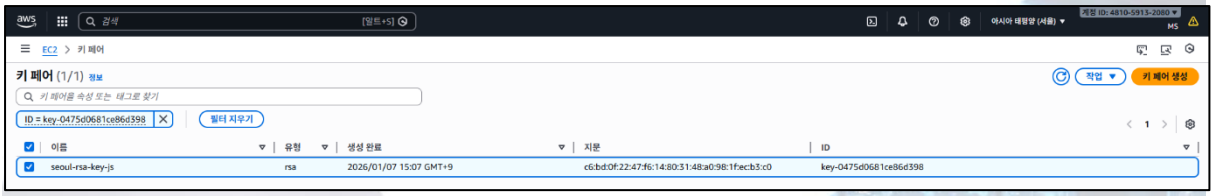
```
# ============================================================
# Keypair 모듈 - SSH 키 페어 생성
# EC2 인스턴스에 SSH 접속하기 위한 공개키/비밀키 쌍을 생성합니다.
# ============================================================
# TLS 비밀키 생성
resource "tls_private_key" "generate" {
  algorithm = var.key_info.algorithm
  rsa_bits = var.key_info.rsa_bits
}
# AWS 키 페어 등록
resource "aws_key_pair" "aws" {
  public_key = tls_private_key.generate.public_key_openssh
  key_name = "${var.region_name}-${lower(var.key_info.algorithm)}-key-js"
}
```

[코드 27] 키페어 설정 코드_ 📂 **modules/security-group/main.tf**

```
key_info:
  # 암호화 알고리즘 (RSA 방식)
  algorithm: "RSA"
  # RSA 키 비트수 (2048 비트 = 표준 보안 수준)
  rsa_bits: 2048
```

[코드 28] 키페어 변수 관리 코드_ 📂 **seoul/variables.yml**

[구축 결과]



[그림 30] 키페어 생성 결과

Copyright ⓒ 2025 Team_JS. All rights reserved.

3.9. 보안 및 트래픽 제어

3.9.1. 민감 정보 변수화 및 보안 관리

[설계 의도 요약]

환경변수를 통해 전달되는 민감 정보가 인프라 구성 과정에서 안전하게 사용되도록 하기 위함.
이를 위해 데이터베이스 비밀번호를 Terraform 변수로 분리하고, 코드 내 하드코딩을 원천적으로 차단함.
또한 YAML 기반 설정 값과 환경변수 값을 병합하여 구조적 설정 관리와 보안성을 동시에 확보함.
그 결과 Terraform 코드가 외부에 공개되더라도 민감 정보 노출 없이 안전한 인프라 구성이 가능하도록 함.

[관련 요청사항 번호: S-02]

항목	내용	
구성 대상	RDS DB Password	WordPress DB Password
변수 명칭	rds_password	wordpress_db_password
데이터 타입	string	
보안 옵션	sensitive = true	
값 주입 방식	환경 변수 (TF_VAR_)	
병합 방식	locals 블록에서 YAML 설정 값과 merge	
적용 리소스	aws_db_instance, WordPress DB 설정	
목적	민감정보 외부 격리 및 설정 구조 일관성 유지	

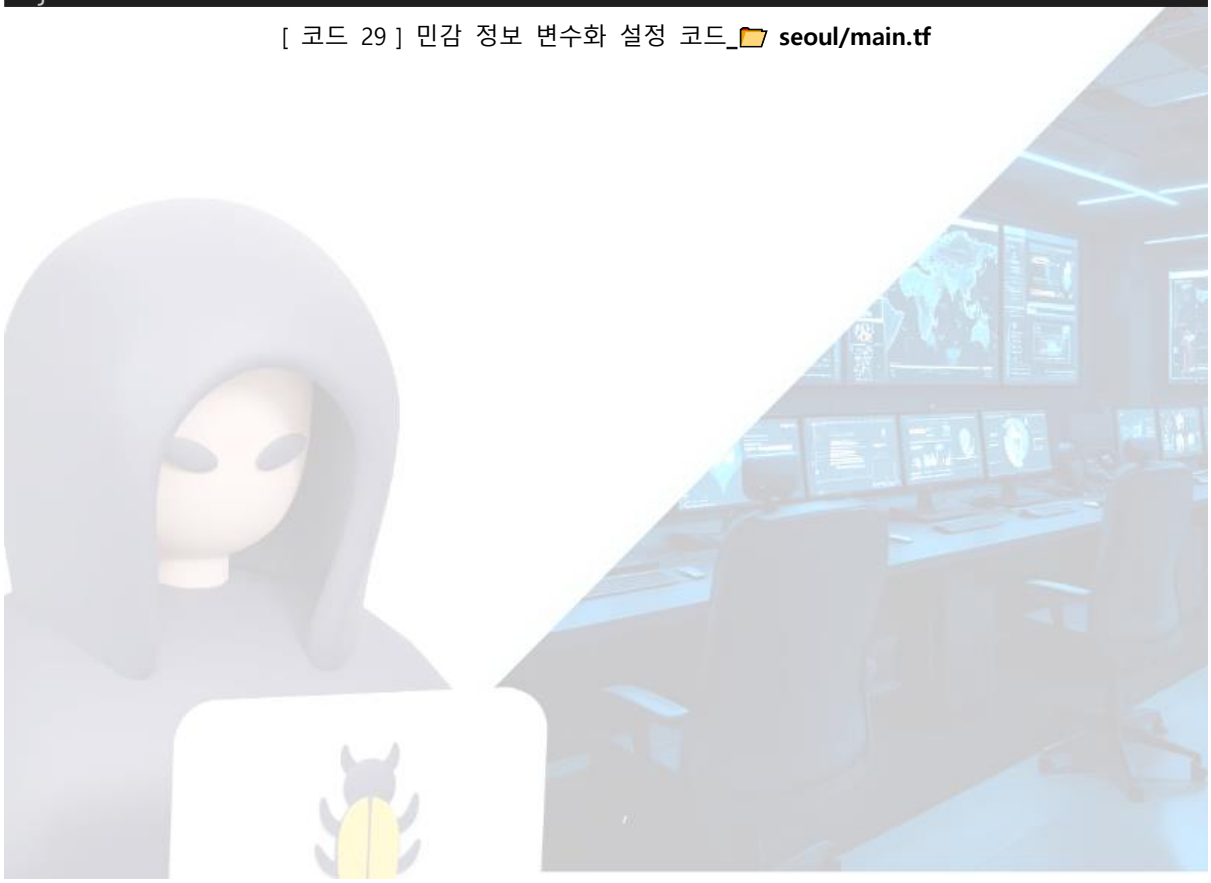
[표 19] 민감 정보 변수화 설정 정보

Copyright ⓒ 2025 Team_JS. All rights reserved.

[상세 구현 코드]

```
locals {
  # 비밀번호 병합: 환경변수에서 읽어서 사용 (보안)
  # 환경변수는 Terraform 변수로 전달됨 (TF_VAR_ 접두사 또는 -var 옵션)
  # load-env.ps1 스크립트가 RDS_PASSWORD 를 TF_VAR_rds_password 로 자동 변환
  # ⚠ 환경변수 필수: RDS_PASSWORD 환경변수가 설정되어 있어야 합니다 (varia-
bles.tf 에서 validation)
  db_instance_merged = merge(
    local.yaml_vars.db_instance,
    { password = var.rds_password }
  )

  # ⚠ 환경변수 필수: WORDPRESS_DB_PASSWORD 환경변수가 설정되어 있어야 합니다
(variables.tf 에서 validation)
  db_config_merged = {
    for k, v in local.yaml_vars.db_config : k => merge(
      v,
      k == "wordpress" ? {
        db_password = var.wordpress_db_password
      } : {}
    )
  }
```

[코드 29] 민감 정보 변수화 설정 코드_📂 **seoul/main.tf**



Copyright ⓒ 2025 Team_JS. All rights reserved.

3.9.2. 환경변수 점검을 통한 보안 검증

[설계 의도 요약]

환경변수 기반으로 전달되는 민감 정보가 배포 과정에서 누락되거나 오설정되는 상황을 방지하고자 함.

이를 위해 PowerShell 스크립트를 활용하여 필수 환경변수 설정 여부를 사전에 점검하도록 구성함.

또한 실제 값은 출력하지 않고 길이만 확인하여 점검 과정에서도 민감 정보 노출을 차단함.

그 결과 배포 안정성과 운영 보안성을 동시에 확보함.

[관련 요청사항 번호: S-02]

항목	내용
구성 대상	환경변수 점검 스크립트
파일명	check-env.ps1
점검 항목	RDS / WordPress DB 비밀번호
출력 방식	값 미출력(길이만 표시)
활용 시점	Terraform apply 실행 전

[표 20] 환경변수 점검을 통한 보안 검증 설정 정보

Copyright ⓒ 2025 Team_JS. All rights reserved.

[상세 구현 코드]

```
# RDS Password Check
if ($env:RDS_PASSWORD) {
    Write-Host "[OK] RDS_PASSWORD: Set" -ForegroundColor Green
} else {
    Write-Host "[FAIL] RDS_PASSWORD: Not set" -ForegroundColor Red
}

Write-Host ""

# WordPress DB Password Check
if ($env:WORDPRESS_DB_PASSWORD) {
    Write-Host "[OK] WORDPRESS_DB_PASSWORD: Set" -ForegroundColor Green
} else {
    Write-Host "[FAIL] WORDPRESS_DB_PASSWORD: Not set" -ForegroundColor Red
}

Write-Host ""

# Terraform Variables Check (TF_VAR_ prefix)
if ($env:TF_VAR_rds_password) {
    Write-Host "[OK] TF_VAR_rds_password: Set" -ForegroundColor Green
} else {
    Write-Host "[INFO] TF_VAR_rds_password: Not set (optional)" -Foreground-
Color Yellow
}

if ($env:TF_VAR_wordpress_db_password) {
    Write-Host "[OK] TF_VAR_wordpress_db_password: Set" -ForegroundColor Green
} else {
    Write-Host "[INFO] TF_VAR_wordpress_db_password: Not set (optional)" -
ForegroundColor Yellow
}
```

[코드 30] 환경변수 점검 스크립트_📂 **seoul/check-env.ps1**

Copyright ⓒ 2025 Team_JS. All rights reserved.

3.9.3. WAF 정책 및 규칙 설정

[설계 의도 요약]

애플리케이션 계층(L7)에서 발생하는 웹 공격을 실시간으로 탐지하고 차단함. 로드밸런서(ALB)에 유입되는 트래픽 중 SQL 인젝션, 교차 사이트 스크립팅(XSS) 등 악성 패턴을 필터링하여 백엔드 서버에 도달하기 전에 무효화함으로써 서비스의 안정성과 데이터 무결성을 보호함.

[관련 요청사항 번호: F-04, S-01]

항목	내용	
구성 대상	WAFv2 Web ACL	
WAF 명칭	seoul-waf-js	
Scope / 연동 대상	Regional / seoul-loadbalance-js (ALB)	
적용 규칙 1	SQLi-Rule	AWS Managed (SQLiRuleSet)
적용 규칙 2	BruteForce-Rule	Rate-Based Rule
적용 규칙 3	WordPress-Rule	AWS Managed (WordPressRuleSet)
적용 규칙 4	Common-Rule	AWS Managed (CommonRuleSet)
기본 액션	Allow (검증된 정상 요청 통과)	
설명	L7 계층 보안 강화를 통한 고가용성 웹 보안 인프라 완성	

[표 21] WAF 설정 정보

[상세 구현 코드]

```
# ============================================================
# ALB = WAF 연결
# Application Load Balancer 에 WAF Web ACL 적용
# ============================================================
resource "aws_wafv2_web_acl_association" "this" {
  count        = var.enable_waf ? 1 : 0
  resource_arn = aws_lb.this.arn
  web_acl_arn  = var.waf_acl_arn
}
```

[코드 31] LB에 WAF 연결 코드_ 📂 **modules/load-balancer/main.tf**

Copyright ⓒ 2025 Team_JS. All rights reserved.

[step 1] 단일 Managed Rule Group

```
# ============================================================
# WAF Web ACL
# WordPress 보호를 위한 AWS WAFv2 구성
# ============================================================
resource "aws_wafv2_web_acl" "wp_waf" {
  name        = var.name
  description = var.description
  scope       = var.scope
  # 기본 액션
  default_action {
    dynamic "allow" {
      for_each = var.default_action == "allow" ? [1] : []
      content {}
    }
    dynamic "block" {
      for_each = var.default_action == "block" ? [1] : []
      content {}
    }
  }
# 단일 AWS Managed Rule (SQLi, XSS 등)
  dynamic "rule" {
    for_each = var.managed_rule_name != "" && length(var.managed_rules) == 0 ?
[1] : []
    content {
      name     = var.rule_name
      priority = var.rule_priority
      override_action {
        dynamic "count" {
          for_each = var.rule_override_action == "count" ? [1] : []
          content {}
        }
        dynamic "none" {
          for_each = var.rule_override_action == "none" ? [1] : []
          content {}
        }
      }
      statement {
        managed_rule_group_statement {
          vendor_name = var.managed_rule_vendor
          name        = var.managed_rule_name
        }
      }
      visibility_config {
        cloudwatch_metrics_enabled = var.rule_visibility.cloudwatch_metrics_ena-
bled
        metric_name                = var.rule_visibility.metric_name
        sampled_requests_enabled   = var.rule_visibility.sampled_requests_enabled
      }
    }
  }
```

[코드 32] 단일 Rule Group_ **modules/waf/main.tf**

Copyright ⓒ 2025 Team_JS. All rights reserved.

[step 2] 다중 Managed Rule

```
  dynamic "rule" {
    for_each = var.managed_rules
    content {
      name     = rule.value.rule_name
      priority = rule.value.priority

      override_action {
        dynamic "count" {
          for_each = rule.value.override_action == "count" ? [1] : []
          content {}
        }
        dynamic "none" {
          for_each = rule.value.override_action == "none" ? [1] : []
          content {}
        }
      }
      statement {
        managed_rule_group_statement {
          vendor_name = rule.value.vendor
          name        = rule.value.managed_rule_name
        }
      }
      visibility_config {
        cloudwatch_metrics_enabled = rule.value.visibility.cloudwatch_met-
rics_enabled
        metric_name                = rule.value.visibility.metric_name
        sampled_requests_enabled   = rule.value.visibility.sampled_re-
quests_enabled
      }
    }
  }
```

[코드 33] 다중 Rule Group_ **modules/waf/main.tf**

Copyright ⓒ 2025 Team_JS. All rights reserved.

[step 3] Rate-Based Rule (무차별 로그인 방어)

```
  dynamic "rule" {
   for_each = var.rate_based_rule.enabled ? [1] : []
   content {
     name     = var.rate_based_rule.rule_name
     priority = var.rate_based_rule.rule_priority

     action {
       dynamic "block" {
         for_each = var.rate_based_rule.action == "block" ? [1] : []
         content {}
       }
       dynamic "count" {
         for_each = var.rate_based_rule.action == "count" ? [1] : []
         content {}
       }
     }
     statement {
       rate_based_statement {
         limit              = var.rate_based_rule.limit
         aggregate_key_type = "IP"

         scope_down_statement {
           or_statement {
             dynamic "statement" {
               for_each = var.rate_based_rule.login_paths
               content {
                 byte_match_statement {
                   positional_constraint = "CONTAINS"
                   search_string         = statement.value

                   field_to_match {
                     uri_path {}
                   }

                   text_transformation {
                     priority = 0
                     type     = "LOWERCASE"
                   }}}}}}}}
     visibility_config {
       cloudwatch_metrics_enabled = var.rate_based_rule.visibility.cloud-
watch_metrics_enabled
       metric_name                = var.rate_based_rule.visibility.metric_name
       sampled_requests_enabled   = var.rate_based_rule.visibility.sampled_re-
quests_enabled
     }}}
  visibility_config {
    cloudwatch_metrics_enabled = var.visibility.cloudwatch_metrics_enabled
    metric_name                = var.visibility.metric_name
    sampled_requests_enabled   = var.visibility.sampled_requests_enabled
  }
}
```

[코드 34] Rate-Based Rule _ 📂 **modules/waf/main.tf**

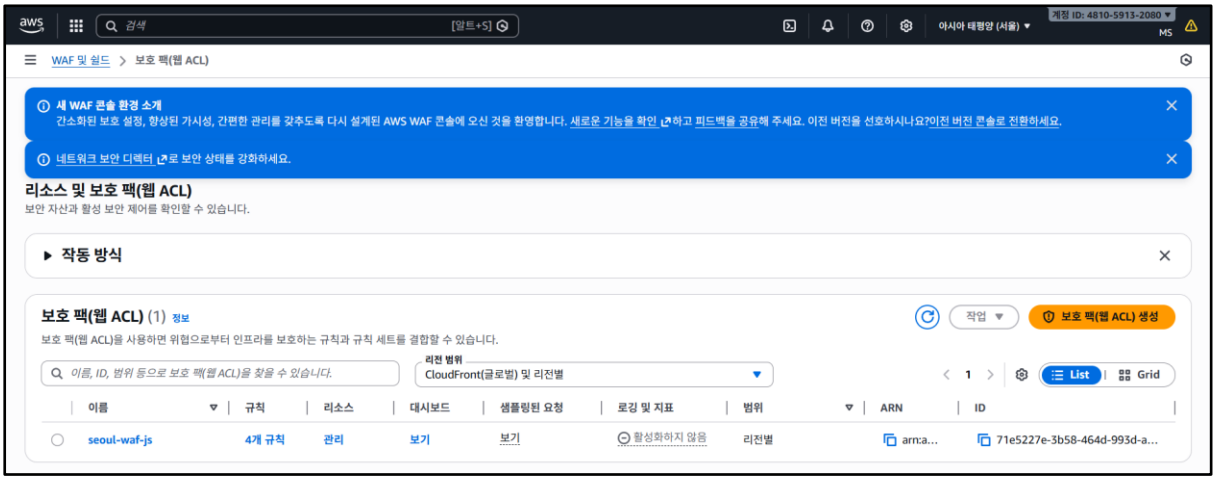
Copyright ⓒ 2025 Team_JS. All rights reserved.

```yaml
waf:
  name                 : "seoul-waf-js"
  description          : "SQL Injection protection for WordPress"
  scope                : "REGIONAL"
  default_action       : "allow"
  rule_name            : "SQLinjection-Rule"
  rule_priority        : 1
  rule_override_action: "none"
  managed_rule_vendor : "AWS"
  managed_rule_name   : "AWSManagedRulesSQLiRuleSet"
  visibility:
    metric_name                : "WordPressWAF"
    cloudwatch_metrics_enabled: true
    sampled_requests_enabled   : true
  rule_visibility:
    metric_name                : "SQLiRuleMetric"
    # cloudwatch_metrics_enabled: true, sampled_requests_enabled: true 동일
  # Managed Rules 리스트 (여러 규칙을 한번에 관리)
  # override_action: "none", vendor: "AWS" 로 동일
  # visibility :
  #     cloudwatch_metrics_enabled: true, sampled_requests_enabled: true 동일
  managed_rules:
    - rule_name               : "SQLi-Rule"
      managed_rule_name       : "AWSManagedRulesSQLiRuleSet"
      priority                : 1
      visibility:
        metric_name     : "SQLiRuleMetric"

    - rule_name               : "WordPress-Rule"
      managed_rule_name       : "AWSManagedRulesWordPressRuleSet"
      priority                : 3
      visibility:
        metric_name     : "WordPressRuleMetric"

    - rule_name               : "Common-Rule"
      managed_rule_name       : "AWSManagedRulesCommonRuleSet"
      priority                : 4
      visibility:
        metric_name    : "CommonRuleMetric"
  # Rate-Based Rule
  # visibility :
  #     cloudwatch_metrics_enabled: true, sampled_requests_enabled: true 동일

  rate_based_rule:
    enabled                   : true
    rule_name                 : "BruteForce-Rule"
    rule_priority             : 2
    limit                     : 15
    evaluation_window_sec     : 300
    login_paths               : [ "/wp-login.php", "/login"]
    Action                    : "block"
    visibility:
      metric_name      : "BruteForceRuleMetric"
```

[코드 35] WAF 변수 관리 코드_ 📂 **seoul/variables.yml**

Copyright ⓒ 2025 Team_JS. All rights reserved.

[구축 결과]



[그림 31] Waf 구축 결과

Copyright ⓒ 2025 Team_JS. All rights reserved.

4. 성능 테스트

4.1. 인프라 구성 결과 검증

[Step1] CPU 부하 발생 확인

top 명령어를 통해 특정 프로세스가 CPU 사용률을 과도하게 점유하고 있음을 확인함. 이로 인해 시스템 전체 CPU 부하가 급증하며, Auto Scaling 정책에서 설정한 CPU 임계치를 초과하는 상황이 발생함.

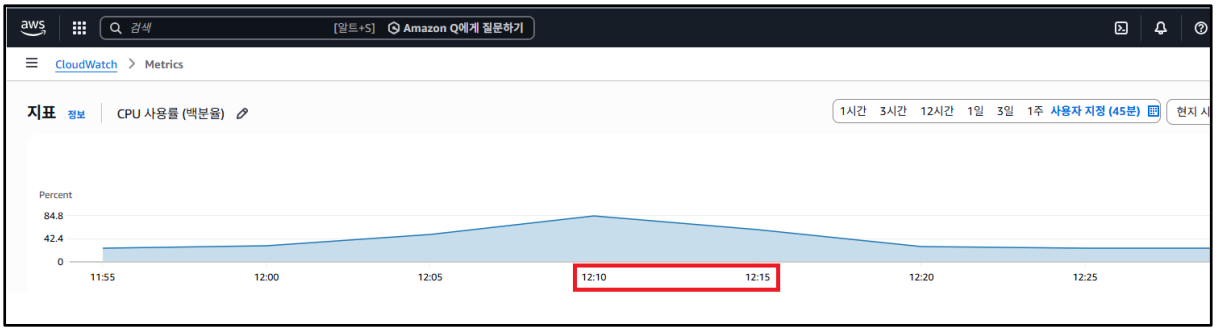
```
top - 03:34:43 up  1:33,  2 users,  load average: 1.00, 1.04, 1.07
Tasks: 117 total,   2 running, 115 sleeping,   0 stopped,   0 zombie
%Cpu(s): 26.7 us, 22.9 sy,  0.0 ni, 49.8 id,  0.0 wa,  0.0 hi,  0.5 si,  0.0 st
MiB Mem :    916.8 total,    321.2 free,    259.0 used,    336.7 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.    489.8 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   4670 root      20   0  221372   1000    908 R  99.7   0.1  28:28.95 yes
      1 root      20   0  172344  17300  10732 S   0.0   1.8   0:01.69 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kthreadd
      3 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 rcu_gp
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 rcu_par_gp
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 slub_flushwq
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 netns
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
     10 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 mm_percpu_wq
     11 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
     12 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
     13 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
     14 root      20   0       0      0      0 S   0.0   0.0   0:00.17 ksoftirqd/0
     15 root      20   0       0      0      0 I   0.0   0.0   0:00.15 rcu_preempt
     16 root      rt   0       0      0      0 S   0.0   0.0   0:00.03 migration/0
     18 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
     19 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/1
     20 root      rt   0       0      0      0 S   0.0   0.0   0:00.04 migration/1
     21 root      20   0       0      0      0 S   0.0   0.0   0:00.16 ksoftirqd/1
     23 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/1:0H-events_highpri
     26 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kdevtmpfs
     27 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 inet_frag_wq
     28 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kauditd
     29 root      20   0       0      0      0 S   0.0   0.0   0:00.00 khungtaskd
     30 root      20   0       0      0      0 S   0.0   0.0   0:00.00 oom_reaper
     32 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 writeback
```

[그림 32] top 명령어 실행 결과

[Step2] CPU 부하 해소에 따른 Scale-in 동작 확인

CPU 부하를 유발하던 프로세스를 종료한 후, CloudWatch CPU 사용률 지표가 정상 범위로 회복됨을 확인함. 이에 따라 Auto Scaling Group이 인스턴스 수를 자동으로 감소시키는 Scale-in 동작을 정상 수행함.

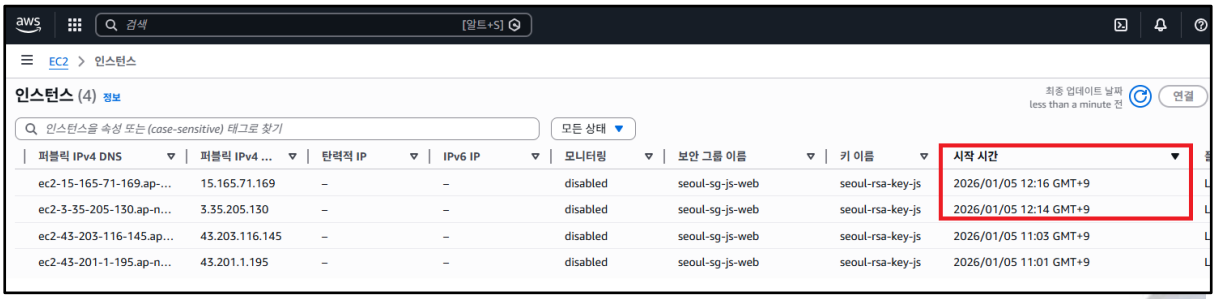
Copyright ⓒ 2025 Team_JS. All rights reserved.



[그림 33] CloudWatch > Metrics 의 CPU 사용률 그래프

[Step3] 지속 부하 발생 시 자동 확장 검증

CPU 부하가 지속적으로 발생하는 상황에서 Auto Scaling 정책이 즉시 트리거되었으며, 신규 EC2 인스턴스 2대가 추가 생성되는 Scale-out 동작이 수행됨을 확인함.



[그림 34] CPU 부하에 따른 자동 확장(Scale-out) 수행 결과

Copyright ⓒ 2025 Team_JS. All rights reserved.

4.2.WAF로 공격 차단 및 로그 확인

4.2.1. BruteForce 차단

[공격 코드]

wp-login.php 엔드포인트를 대상으로 짧은 시간 내 다수의 HTTP GET 요청을 반복 전송하는 스크립트를 실행함.

이를 통해 AWS WAF에 설정된 Rate-based Rule(5분 내 15회 초과 요청 차단) 이 정상적으로 동작하는지 검증함.

```
# ============================================================
# 3. Rate-based Brute Force Attack Test
# (Blocked if 15+ requests within 5 minutes)
# ============================================================
Write-Host "[3] Rate-based Brute Force Attack Test" -ForegroundColor Yellow
Write-Host "  Path: /wp-login.php (Blocked if 15+ requests in 5 min)" -ForegroundColor Gray
Write-Host "  Sending 16 requests quickly..." -ForegroundColor Gray

$bruteforceUrl = $targetUrl + "/wp-login.php"
$blockedCount = 0
$allowedCount = 0

for ($i = 1; $i -le 20; $i++) {
    Write-Host "  Request #$i..." -NoNewline -ForegroundColor Gray

    try {
        $response = Invoke-WebRequest -Uri $bruteforceUrl -Method GET -TimeoutSec 5 -UseBasicParsing
        Write-Host " Allowed (HTTP $($response.StatusCode))" -ForegroundColor Green
        $allowedCount++
    }
    catch {
        if ($_.Exception.Response.StatusCode.value__ -eq 403) {
            Write-Host " Blocked (403 Forbidden) [OK]" -ForegroundColor Red
            $blockedCount++
        }
        else {
            Write-Host " Other response: $($_.Exception.Message)" -ForegroundColor Yellow
        }
    }

    # Short delay for rapid consecutive requests
    Start-Sleep -Milliseconds 200
}
```

[그림 35] BruteForece 공격 코드

Copyright ⓒ 2025 Team_JS. All rights reserved.

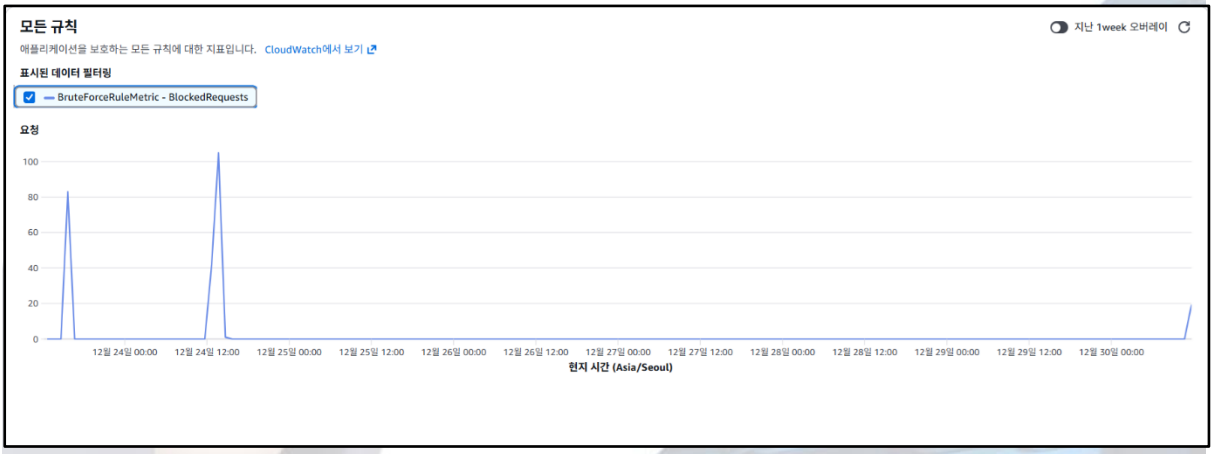
[공격 결과]

초기 요청은 정상적으로 처리되었으나, 허용된 요청 횟수(15회)를 초과한 이후부터 서버 응답이 403 Forbidden으로 변경되며 해당 IP의 요청이 차단되는 것을 확인함.

```
========================================
WAF Attack Test Started
Target: http://www.js.it-edu.org
========================================

[3] Rate-based Brute Force Attack Test
  Path: /wp-login.php (Blocked if 15+ requests in 5 min)
  Sending 16 requests quickly...
  Request #1... Allowed (HTTP 200)
  Request #2... Allowed (HTTP 200)
  Request #3... Allowed (HTTP 200)
  Request #4... Allowed (HTTP 200)
  Request #5... Allowed (HTTP 200)
  Request #6... Allowed (HTTP 200)
  Request #7... Allowed (HTTP 200)
  Request #8... Allowed (HTTP 200)
  Request #9... Allowed (HTTP 200)
  Request #10... Allowed (HTTP 200)
  Request #11... Allowed (HTTP 200)
  Request #12... Allowed (HTTP 200)
  Request #13... Allowed (HTTP 200)
  Request #14... Allowed (HTTP 200)
  Request #15... Blocked (403 Forbidden) [OK]
  Request #16... Blocked (403 Forbidden) [OK]
  Request #17... Blocked (403 Forbidden) [OK]
  Request #18... Blocked (403 Forbidden) [OK]
  Request #19... Blocked (403 Forbidden) [OK]
  Request #20... Blocked (403 Forbidden) [OK]
```

[그림 36] BruteForece 공격결과

[로그 확인]



[그림 37] BrutoForce 로그 확인

Copyright ⓒ 2025 Team_JS. All rights reserved.

4.2.2. SQLiRule 차단

[공격 코드]

OR 1=1, UNION SELECT, DROP TABLE 등 SQL Injection 공격 패턴이 포함된 쿼리를 HTTP 요청에 삽입하여 웹 서비스로 전송함.

이를 통해 AWS WAF의 SQLi Rule 이 악성 SQL 페이로드를 탐지하는지 검증함.

```
# ============================================================
# 1. SQL Injection Attack Test
# ============================================================
Write-Host "[1] SQL Injection Attack Test" -ForegroundColor Yellow

$sqlInjectionPayloads = @(
    '?id=1'' OR ''1''=''1',
    '?id=1'' UNION SELECT NULL--',
    '?id=1; DROP TABLE users--',
    '?id=1'' OR 1=1--',
    '/wp-login.php?user=admin''--'
)

foreach ($payload in $sqlInjectionPayloads) {
    $attackUrl = $targetUrl + $payload
    Write-Host "  Attack: $attackUrl" -ForegroundColor Gray

    try {
        $response = Invoke-WebRequest -Uri $attackUrl -Method GET -TimeoutSec 10 -UseBasicParsing
        Write-Host "  [X] Not blocked (HTTP $($response.StatusCode))" -ForegroundColor Red
    }
    catch {
        if ($_.Exception.Response.StatusCode.value__ -eq 403) {
            Write-Host "  [OK] Blocked by WAF (403 Forbidden)" -ForegroundColor Green
        }
        else {
            Write-Host "  [!] Response: $($_.Exception.Message)" -ForegroundColor Yellow
        }
    }
    Start-Sleep -Seconds 1
}
```

[그림 38] SQL 공격 코드

Copyright ⓒ 2025 Team_JS. All rights reserved.

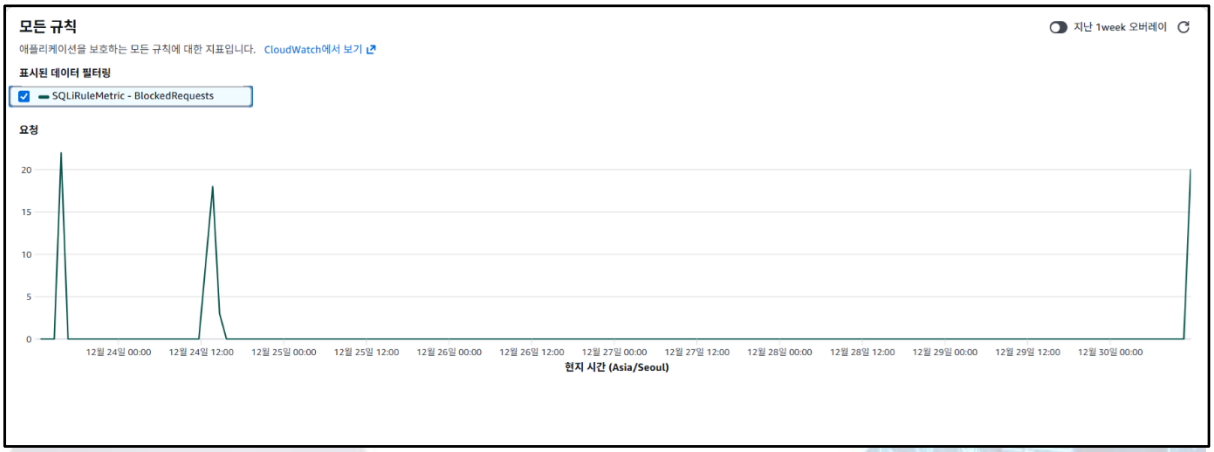
[공격 결과]

SQL Injection 패턴이 포함된 요청이 서버에서 처리되지 않고 403 Forbidden 응답으로 차단되는 것을 확인함.

```
========================================
WAF Attack Test Started
Target: http://www.js.it-edu.org
========================================

[1] SQL Injection Attack Test
  Attack: http://www.js.it-edu.org?id=1' OR '1'='1
  [OK] Blocked by WAF (403 Forbidden)
  Attack: http://www.js.it-edu.org?id=1' UNION SELECT NULL--
  [OK] Blocked by WAF (403 Forbidden)
  Attack: http://www.js.it-edu.org?id=1; DROP TABLE users--
  [OK] Blocked by WAF (403 Forbidden)
  Attack: http://www.js.it-edu.org?id=1' OR 1=1--
  [OK] Blocked by WAF (403 Forbidden)
  Attack: http://www.js.it-edu.org/wp-login.php?user=admin'--
  [OK] Blocked by WAF (403 Forbidden)
```

[그림 39] SQL 공격 결과

[로그 확인]



[그림 40] SQL 로그 확인

Copyright ⓒ 2025 Team_JS. All rights reserved.

4.2.3. CommonRule 차단

[공격 코드]

WordPress 환경에서 자주 악용되는 경로(wp-admin/admin-ajax.php, wp-config.php, revslider/db.php 등)에 대해 직접 접근을 시도하는 요청을 전송함.

이를 통해 AWS WAF의 Common Rule Set 이 WordPress 특화 공격 패턴을 탐지·차단하는지 검증함.

```
# ============================================================
# 2. WordPress Specific Attack Test
# ============================================================
Write-Host "[2] WordPress Specific Attack Test" -ForegroundColor Yellow

$wordpressAttacks = @(
    "/wp-admin/admin-ajax.php?action=revslider_show_image&img=../wp-config.php",
    "/wp-content/plugins/revslider/temp/update_extract/revslider/db.php",
    "/wp-login.php?action=lostpassword",
    "/xmlrpc.php",
    "/wp-includes/wp-version.php"
)

foreach ($path in $wordpressAttacks) {
    $attackUrl = $targetUrl + $path
    Write-Host "  Attack: $attackUrl" -ForegroundColor Gray

    try {
        $response = Invoke-WebRequest -Uri $attackUrl -Method GET -TimeoutSec 10 -UseBasicParsing
        Write-Host "  [X] Not blocked (HTTP $($response.StatusCode))" -ForegroundColor Red
    }
    catch {
        if ($_.Exception.Response.StatusCode.value__ -eq 403) {
            Write-Host "  [OK] Blocked by WAF (403 Forbidden)" -ForegroundColor Green
        }
        else {
            Write-Host "  [!] Response: $($_.Exception.Message)" -ForegroundColor Yellow
        }
    }
    Start-Sleep -Seconds 1
}
```

[그림 41] CommonRule 공격 코드

Copyright ⓒ 2025 Team_JS. All rights reserved.

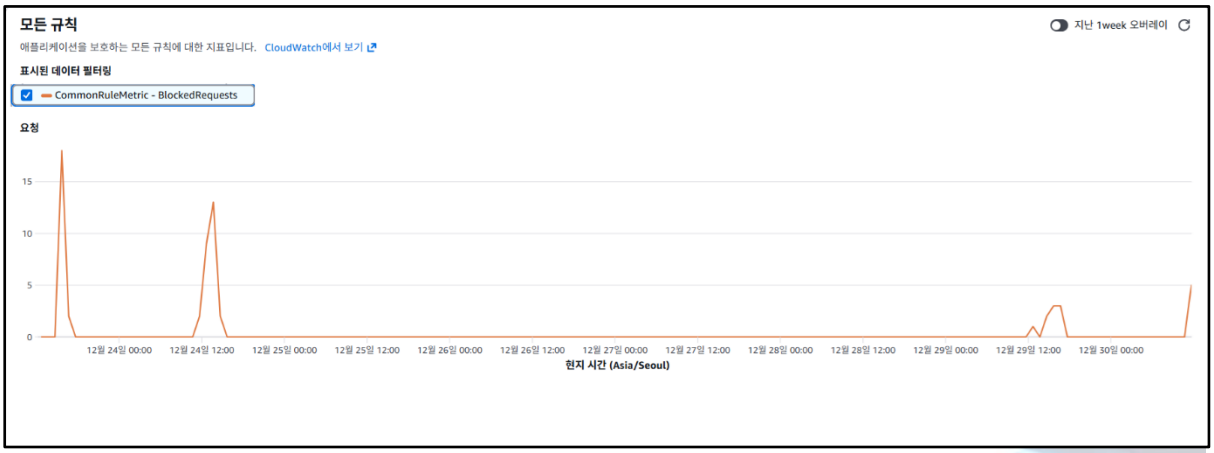
[공격 결과]

WordPress 취약 경로에 대한 접근 요청이 403 Forbidden 응답으로 차단되는 것을 확인함.

```
====================================
WAF Attack Test Started
Target: http://www.js.it-edu.org
====================================

[2] WordPress Specific Attack Test
 Attack: http://www.js.it-edu.org/wp-admin/admin-ajax.php?action=revslider_show_image&img=../wp-config.php
 [OK] Blocked by WAF (403 Forbidden)
 Attack: http://www.js.it-edu.org/wp-content/plugins/revslider/temp/update_extract/revslider/db.php
 [!] Response: 원격 서버에서 (404) 찾을 수 없음 오류를 반환했습니다.
 Attack: http://www.js.it-edu.org/wp-login.php?action=lostpassword
 [OK] Blocked by WAF (403 Forbidden)
 Attack: http://www.js.it-edu.org/xmlrpc.php
 [OK] Blocked by WAF (403 Forbidden)
 Attack: http://www.js.it-edu.org/wp-includes/wp-version.php
 [!] Response: 원격 서버에서 (404) 찾을 수 없음 오류를 반환했습니다.
```

[그림 42] CommonRule 공격 결과

[로그 확인]



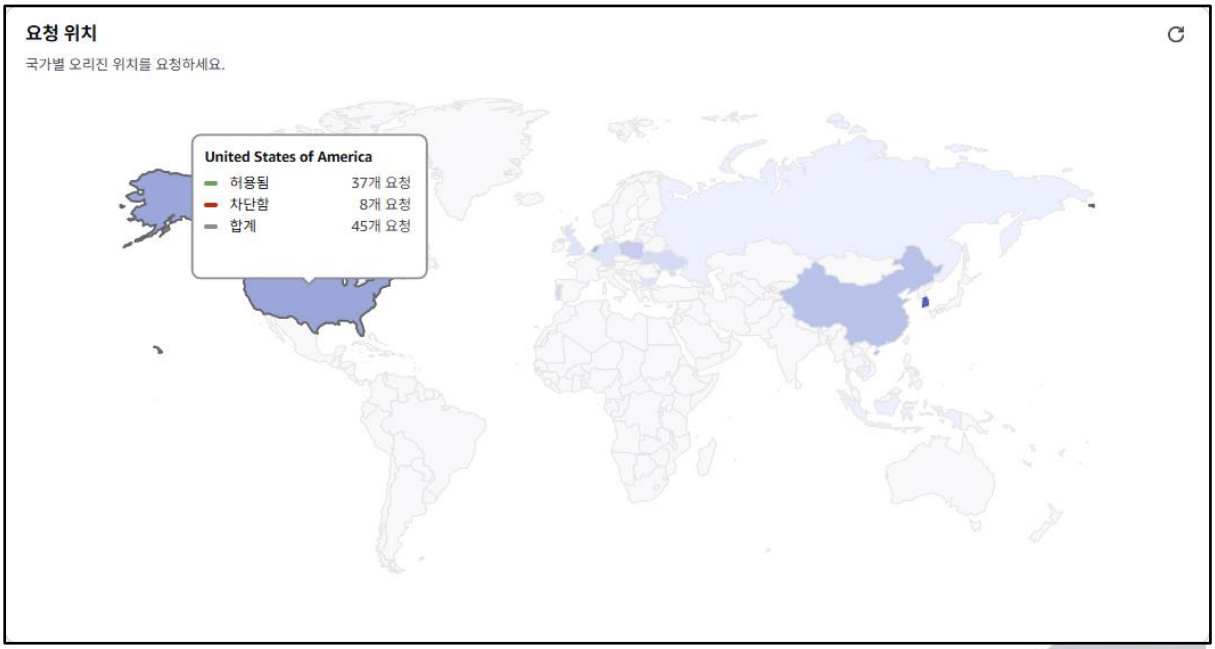
[그림 43] CommonRule 로그 확인

Copyright ⓒ 2025 Team_JS. All rights reserved.

4.2.4. 대시보드 확인

[국가별 공격]

Application Load Balancer가 전 세계에서 접근 가능한 공개 엔드포인트로 구성됨에 따라, 다양한 국가의 IP에서 공격 시도가 유입되고 있음을 확인함.

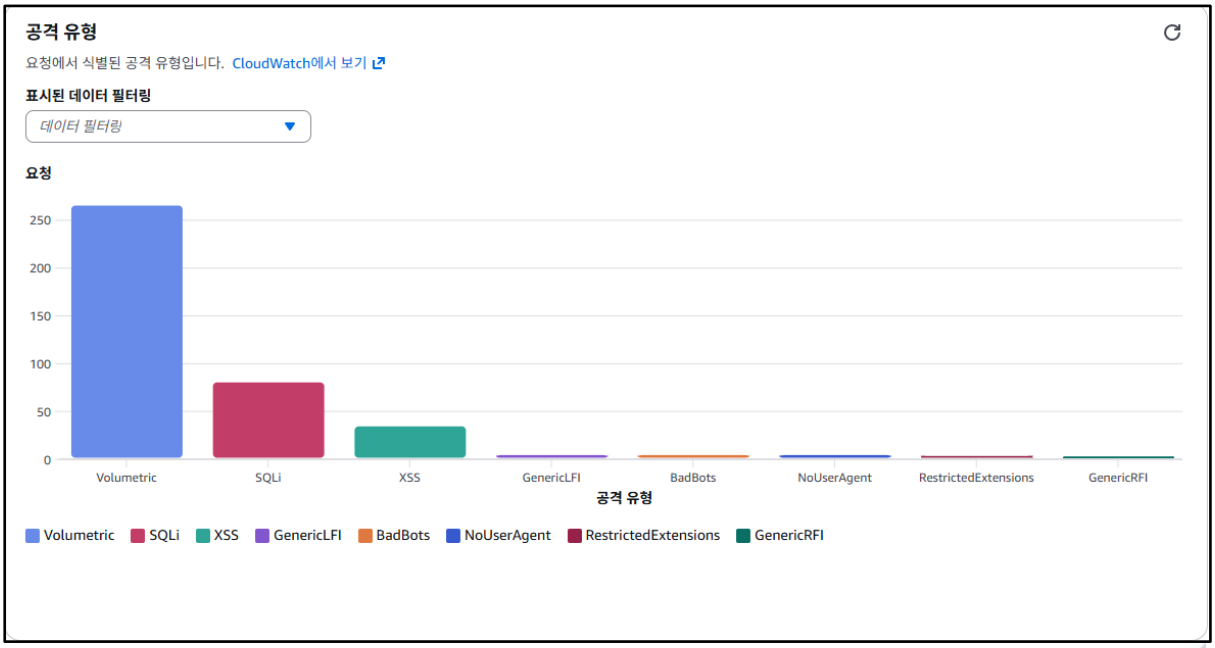


[그림 44] WAF 및 쉴드 > 보호 팩(웹 ACL) > seoul-waf-js > 대시보드의 국가별 공격

Copyright ⓒ 2025 Team_JS. All rights reserved.

[공격 유형]

WAF 대시보드를 통해 Rate-based 공격, SQL Injection, WordPress 관련 공격 등 다양한 공격 유형이 실시간으로 탐지·차단되고 있음을 확인함.



[그림 45] WAF 및 쉴드 > 보호 팩(웹 ACL) > seoul-waf-js > 대시보드에서의 공격유형

Copyright ⓒ 2025 Team_JS. All rights reserved.

5. 문제점 및 개선사항

본 프로젝트를 통해 Terraform 기반의 AWS WordPress 고가용성·보안 인프라를 구축하였으나, 구현 과정 및 운영 관점에서 일부 한계점과 개선이 필요한 요소가 확인되었습니다.
이에 본 장에서는 현재 구조의 문제점을 분석하고, 향후 개선 방향을 제시합니다.

5.1. 수동 작업 요소 존재

문제점

- WordPress 초기 설정 및 AMI 생성 과정이 수동으로 수행됨
- RDS 초기 구성 및 스냅샷 생성에 운영자 개입 필요
  → 인프라 재현성 저하 및 휴먼 에러 가능성 존재

개선 방향

- User Data 및 Cloud-init을 활용한 초기 설정 자동화
- Packer 기반 AMI 자동 생성 파이프라인 도입
- RDS 초기화 스크립트 자동화

5.2. 비용 문제

문제점

- ALB, ASG(최소 2대), WAF, RDS 상시 운영 구조
  → 소규모 환경에서는 비용 비효율 발생 가능

개선 방향

- 개발/운영 환경 분리
- 시간 기반 Auto Scaling 적용
- WAF Rule 및 로그 보존 정책 최적화
- RDS 스케줄링 활용

Copyright ⓒ 2025 Team_JS. All rights reserved.

5.3. 보안 및 운영 자동화 측면의 한계

문제점

- HTTPS 미적용
- 위협 탐지 및 중앙 로그 관리 체계 미흡
- Terraform 배포가 로컬 환경에 의존

개선 방향

- ACM 기반 HTTPS 적용
- GuardDuty, Security Hub 연동
- CloudWatch 기반 로그 중앙화
- GitHub Actions 기반 Terraform CI/CD 구축

5.4. 문제점 및 개선사항 종합 정리

구분	현재 상태 (As-Is)	개선 방향 (To-Be)
배포	로컬 환경 수동 배포	CI/CD 기반 자동 배포
보안	WAF + SG 중심	HTTPS + IDS/IPS 연동
운영	개별 리소스 모니터링	통합 대시보드 운영
비용	상시 리소스 유지	환경별 비용 최적화

[표 22] 문제점 및 개선사항 종합 정리

Copyright ⓒ 2025 Team_JS. All rights reserved.